Building-up constructions, cohomology, and Lean 4 formalization of self-dual codes over finite fields

Jae-Hyun Baek
Department of Artificial Intelligence and
Institute for Mathematical and Data Sciences
Sogang University, Seoul, Korea

Jon-Lark Kim
Department of Mathematics and
Institute for Mathematical and Data Sciences
Sogang University, Seoul, Korea

March 21, 2026

Abstract

We develop a unified framework connecting the building-up construction of self-dual codes over $\mathbb{F}_q$ where $q \equiv 1 \pmod 4$ with the cohomological theory governing their existence, structure, and classification. The condition that -1 is a square is shown to be the common algebraic root underlying both the étale-cohomological triviality and the self-orthogonality mechanism of the building-up construction. As applications, we give the complete classification of self-dual codes over GF(5) for lengths $2, 4, 6, 8$, including a two-type classification for $[6, 3]$ codes and a three-type classification for the 53 inequivalent $[8, 4]$ codes by weight enumerator. These structural statements are accompanied by a Lean 4 formalization of the algebraic core.

1 Introduction

Coding theory is the study of reliable communication. It was started by Claude Shannon from his seminal paper [1]. Since then, various kinds of linear codes have been studied. In particular, famous linear codes including the binary Hamming code of length 8, the binary Golay code of length 24, and the ternary Golay code of length 12 are all self-dual, that is, $C = C^{\perp}$. Thus the classification or construction of self-dual codes has been one of the most active research problems [2]. Self-dual codes have wide connection with several mathematical

areas including t-design theory [3], group theory [5], number theory [4], [5], [6]. Furthermore, they have applications in quantum error-correcting codes [7].

Kim [8] introduced "building-up construction" in order to construct many new binary self-dual codes. This construction starts from a binary self-dual code of length $2n$ and then add a vector of length $2n$ of odd weight to construct a binary self-dual code of length $2n+2$. Kim–Lee [9, 10] generalized this method to construct self-dual codes over finite fields $\mathbb{F}_q$, where q is even or $q \equiv 1 \pmod 4$. In practice, this is the main constructive source of examples over split fields. However, the formula for the new generator rows is often presented as an ad hoc correction term. One of the central points of the present paper is that these correction terms are naturally organized by an isotropic line in a hyperbolic plane, and that the forward construction admits a precise row-wise and family-wise characterization in those terms.

More specifically, let q be an odd prime power with $q \equiv 1 \pmod 4$. Over $\mathbb{F}_q$, self-dual codes are maximal totally isotropic subspaces for the standard Euclidean bilinear form on $\mathbb{F}_q^{2n}$. In this split case the elementary identity $c^2 = -1$ is the basic algebraic input behind both the classical building-up construction and the structural description of the ambient quadratic space: it produces an isotropic line in $\mathbb{F}_q^2$, identifies the Euclidean plane with a hyperbolic plane, and makes the norm form $x^2 + y^2$ split. The purpose of this paper is to make that common mechanism explicit and to push it simultaneously in three directions: constructive building-up, cohomological interpretation, and machine-checked verification.

We establish that each generator row of the building-up admits the isotropic line decomposition $r_i = -y_i \cdot (1, c, 0, \ldots, 0) + (0, 0, g_i)$, where $(1, c)$ spans an isotropic line in the hyperbolic plane. This decomposition provides a precise geometric interpretation of the correction terms in the building-up construction. We also give a constructive norm-form witness: once $c^2 = -1$ and $2 \neq 0$, every $a \in \mathbb{F}_q$ can be written explicitly as $a = x^2 + y^2$, which in turn yields extension vectors of prescribed norm.

The cohomological viewpoint explains why the split case is the correct one. Since $q \equiv 1 \pmod 4$, the element -1 is a square in $\mathbb{F}_q^\times$; equivalently, the Euclidean form is hyperbolic in every even dimension, and there is no arithmetic obstruction to the existence of self-dual codes. This viewpoint also interfaces naturally with monomial equivalence, maximal isotropic subspaces, and the combinatorics of Tits buildings [17, 19].

In the present paper, we use the above remarks not only as background motivation but as an organizing principle for the algebra carried out in the building-up section and for the small-length classification over GF(5).

Our main contributions are as follows.

(1) We show that the building-up rows are controlled by an isotropic line and obtain row-wise and family-wise characterization theorems for split build families, together with uniqueness of the child over a fixed parent (Section 3).

(2) We isolate a Lean-verified algebraic core of the split building-up construction: hyper-

bolic realization, constructive norm witnesses, seed-code self-duality, the theorem-level equivalence between row-wise orthogonality and Gram-matrix vanishing, and the split forward construction itself (Section 3).

(3) We develop a normalization theory for the reverse direction: nonzero head normalization, invariance of the generated code under row scaling and row permutation, pivot normalization, pivot-tail packages, and pure-tail representatives up to code equivalence (Section 3).

(4) We relate the split algebra to the broader cohomological picture through the existence of fourth roots of unity, vanishing Brauer obstructions, monomial orbit structure, and counting formulas for maximal isotropic subspaces; this discussion culminates in Theorem 3.36.

(5) As a concrete application, we classify self-dual codes over GF(5) of lengths $2, 4, 6, 8$, including the two-type classification of $[6, 3]$ codes and the three-type classification of the 53 inequivalent $[8, 4]$ codes by weight enumerator (Section 4).

(6) The integrated Lean 4 development formalizes 247 theorems with zero `sorry` in a single file, including the forward split theory, the normalization front-end for the reverse direction, the head-alignment step, and the reverse theorem up to split-isometric equivalence (Section 5).

The paper is organized as follows. Section 2 fixes notation and recalls the standard facts on self-dual codes, split quadratic forms, and hyperbolic planes that are used throughout. Section 3 develops the algebraic core of the split building-up construction, proves the characterization and normalization theorems used later, and includes the cohomological and building-theoretic discussion culminating in Theorem 3.36. Section 4 gives the constructive classification over GF(5). Section 5 summarizes the formalization, and Section 6 explains how the reverse theorem closes up to split-isometric equivalence and how this compares with the classical Lee–Kim wording.

2 Preliminaries

2.1 Self-Dual Codes over Finite Fields

Throughout the paper, q denotes an odd prime power satisfying $q \equiv 1 \pmod 4$, and $\mathbb{F}_q$ denotes the field with q elements. We equip $\mathbb{F}_q^n$ with the Euclidean bilinear form

$$\langle u, v \rangle := \sum_{i=1}^{n} u_i v_i.$$

For a linear code $C \subseteq \mathbb{F}_q^n$, we write

$$C^\perp := \{v \in \mathbb{F}_q^n : \langle v, c \rangle = 0 \text{ for all } c \in C\}.$$

Definition 2.1 (self-dual code) A linear code $C \subseteq \mathbb{F}_q^n$ is **self-dual** if $C = C^\perp$. Equivalently, C is a maximal totally isotropic subspace of $(\mathbb{F}_q^n, \langle \cdot, \cdot \rangle)$.

If C is self-dual, then necessarily n is even and $\dim C = n/2$. In particular, a self-dual code of length $2k$ is a Lagrangian subspace of the split quadratic space $\mathbb{F}_q^{2k}$ [11].

Proposition 2.1 (Systematic form criterion) *Let $C \subseteq \mathbb{F}_q^{2k}$ be generated by a matrix $G = [I_k \mid A]$ with $A \in M_k(\mathbb{F}_q)$. Then C is self-dual if and only if*

$$AA^T = -I_k.$$

Proof. The rows of G span a k-dimensional code. Hence self-duality is equivalent to pairwise orthogonality of the rows. The Gram matrix GG^T of the rows is

$$GG^T = I_k + AA^T = O,$$

so the orthogonality condition is exactly $AA^T = -I_k$. □

Definition 2.2 (Monomial Equivalence) Two codes $C_1, C_2 \subseteq \mathbb{F}_q^n$ are **monomially equivalent** if C_2 is obtained from C_1 by a permutation of coordinates followed by multiplication of each coordinate by a nonzero scalar. The monomial group is

$$\mathrm{Mon}(n, q) = (\mathbb{F}_q^\times)^n \rtimes S_n,$$

and its action preserves the Euclidean inner product up to the obvious transport of coordinates. In particular, self-duality is invariant under the monomial equivalence.

2.2 Building-up Construction of Binary Self-Dual Codes

In 2001, Kim [8] proposed a building-up construction of binary self-dual codes in order to construct many self-dual codes of larger lengths from a self-dual code of a given length. An explicit form of the building-up construction is given below.

Theorem 2.2 *([8, Theorem 1]) Let G_0 be a generator matrix of a binary self-dual code C_0 of length $2n$, whose rows are $g_1, \ldots, g_n$. Let $\mathbf{x}$ be a binary vector in $\mathbb{F}_2^{2n}$ satisfying $\mathbf{x} \cdot \mathbf{x} = 1$. Suppose that $y_i = \mathbf{x} \cdot g_i$ for $1 \le i \le n$. Then the following matrix*

$$G = \left(\begin{array}{ccc} 1 & 0 & \mathbf{x} \\ \hline y_1 & y_1 & g_1 \\ \vdots & \vdots & \vdots \\ y_n & y_n & g_n \end{array}\right)$$

generates a binary self-dual code C of length $2n + 2$.

Then the converse of Theorem 2.2 was proved in [8] as follows.

Theorem 2.3 *([8, Theorem 2]) Any binary self-dual code C of length $2n$ with minimum distance $d > 2$ is obtained from some self-dual code C_0 of length $2n-2$ (up to equivalence) by the construction in Theorem 2.2.*

In 2012, Chinburg and Zhang [12] proved that up to equivalence, all binary self-dual codes of length at least 4 arise from Hilbert pairings on rings of S-integers of $\mathbb{Q}$.

Theorem 2.4 *([12, Theorem 1.5]) Up to equivalence, all binary self-dual codes of length at least 4 arise from Hilbert pairings on rings of S-integers of $K = \mathbb{Q}$. In fact, each such code arises up to equivalence from infinitely many different subsets S of the places of $K = \mathbb{Q}$.*

In the process of the proof, ChinburgZhang [12] drew the following boxed code.

S-units \ places	W_{p_1}	W_{p_2}	$\cdots$	W_2		$W_{\mathbb{R}}$
p_1	01	00/11		00/11	1	0
p_2	11/00	01		11/11	1	0
$\cdots$	$\cdots$		$\ddots$			$\vdots$
2	11/00			01	1	0
-1	11	11	$\cdots$	11	1	1

The authors [12] showed that (i) every binary self-dual code of length $2n$ can be written in this form, and (ii) by deleting the left most two columns of the boxed matrix and deleting the top row, a smaller self-orthogonal code of length $2n-2$ is also obtained.

We have noticed the similarity between the matrix in Theorem 2.2 and the above boxed matrix. In fact, Theorem 2.2 is to build up binary self-dual codes from the trivial self-dual code of length 2 with generator matrix [1 1] while Chinburg-Zhang's approach is to build down or decompose a given self-dual code into a shorter self-dual code. We return to this comparison later in both ordinary mathematical language and its Lean counterpart.

Theorem 2.5 *Up to code equivalence, Kim's building-up construction for binary self-dual codes and Chinburg–Zhang's boxed Hilbert-pairing construction describe the same recursion.*

Proof. In the boxed form, the distinguished row is $r_0 = (1, 0, x)$ and each successor row has the form

$$r_i = (y_i, y_i, g_i).$$

Over $\mathbb{F}_2$, orthogonality to r_0 forces

$$y_i = \langle x, g_i \rangle,$$

so the boxed family is exactly the Kim family

$$\big(r_0,\ (\langle x, g_i\rangle, \langle x, g_i\rangle, g_i)\big).$$

Deleting the first row and the first hyperbolic coordinate pair recovers the shorter parent family, and rebuilding from a family in boxed/Kim normal form recovers the original family. Thus the Chinburg–Zhang reduction and the Kim extension are inverse descriptions of the same binary self-dual code family, up to the same equivalence operations used in both constructions. The arithmetic formulation is made precise later in Theorem 3.36. □

Listing 1: Lean bundle for Theorem 2.5.

```
theorem boxedFamily_eq_buildRowsBin
    (hY : ∀ i : Fin m, Y i = dot x (G i)) :
    boxedFamily x Y G = buildRowsBin x G

theorem deleteHyperbolicPair_buildRowsBin :
    deleteHyperbolicPair (buildRowsBin x G) = G

theorem rebuild_of_IsBoxedKimFamily
    (hR : IsBoxedKimFamily x R) :
    buildRowsBin x (deleteHyperbolicPair R) = R

theorem sameCode_boxedFamily_buildRowsBin
    (hY : ∀ i : Fin m, Y i = dot x (G i)) :
    SameCode (boxedFamily x Y G) (buildRowsBin x G)

theorem sameCode_rebuild_of_IsBoxedKimFamily
    (hR : IsBoxedKimFamily x R) :
    SameCode (buildRowsBin x (deleteHyperbolicPair R)) R

theorem codeEquiv_rebuild_of_IsBoxedKimFamily
    (hR : IsBoxedKimFamily x R) :
    CodeEquiv (buildRowsBin x (deleteHyperbolicPair R)) R
```

2.3 Building-up Construction of Self-Dual Codes over $\mathbb{F}_q$ ($q \equiv 1 \pmod 4$)

Motivated by the building-up construction of binary self-dual codes, Kim and Lee [9] generalized it to finite fields with q elements where $q \equiv 1 \pmod 4$ as follows.

Theorem 2.6 *([9, Proposition 2.1]) Let $\mathbb{F}_q$ be a finite field with q elements. Assume that $q \equiv 1 \pmod 4$. Let c be in $\mathbb{F}_q$ such that $c^2 = -1$. Let G_0 be a generator matrix of a Euclidean self-dual code C_0 over $\mathbb{F}_q$ of length $2n$, whose rows are $g_1, \ldots, g_n$. Let $\mathbf{x}$ be a vector*

in $\mathbb{F}_q^{2n}$ satisfying $\mathbf{x}\cdot\mathbf{x}=1$. Suppose that $y_i=\mathbf{x}\cdot g_i$ for $1\le i\le n$. Then the following matrix

$$G=\left(\begin{array}{ccc} 1 & 0 & \mathbf{x} \\ \hline -y_1 & cy_1 & g_1 \\ \vdots & \vdots & \vdots \\ -y_n & cy_n & g_n \end{array}\right)$$

generates a self-dual code C over $\mathbb{F}_q$ of length $2n+2$.

Remark 2.7 From this point on we use the equivalent normalization obtained by replacing c with $-c$. Since $(-c)^2=-1$, this only changes the sign convention in the second correction coordinate. Accordingly, our internal formulas use rows of the form $(-y_i,-cy_i,g_i)$.

Then the converse of the above theorem was also stated and proved in [9].

Theorem 2.8 *([9, Proposition 2.2]) Let $\mathbb{F}_q$ be a finite field with q elements. Assume that $q\equiv 1 \pmod 4$. Any Euclidean self-dual code C over $\mathbb{F}_q$ of length $2n$ with minimum weight $d>2$ is obtained from some Euclidean self-dual code C_0 over $\mathbb{F}_q$ of length $2n-2$ (up to permutation equivalence) by the construction method in Theorem 2.6.*

One natural question is the following:

Is there a q-ary type of Chinburg-Zhang's approach for self-dual codes over $\mathbb{F}_q$ for $q\equiv 1 \pmod 4$?

We answer it positively by putting the construction into a split hyperbolic normal form in which both the forward building-up step and the reverse reduction become transparent.

Theorem 2.9 (Split Lagrangian reduction and building-up extension over $\mathbb{F}_q$) *Let q be a prime power with $q\equiv 1 \pmod 4$, and let $K=\mathbb{F}_q$. Fix $c\in K^\times$ such that $c^2=-1$. Let $W=K^{2n}$ be equipped with the standard Euclidean inner product*

$$\langle u,v\rangle=\sum_{i=1}^{2n}u_iv_i.$$

Let $L\subset W$ be a self-dual code, i.e. $L=L^\perp$.

Suppose that, after a monomial equivalence and row operations, L admits a generator matrix of the form

$$M=\begin{pmatrix} 1 & 0 & x \\ -y_1 & -cy_1 & g_1 \\ \vdots & \vdots & \vdots \\ -y_{n-1} & -cy_{n-1} & g_{n-1} \end{pmatrix},$$

where $x\in K^{2n-2}$ and $g_i\in K^{2n-2}$ for $1\le i\le n-1$.

Let $W_0=K^{2n-2}$ and define $L_0:=\langle g_1,\dots,g_{n-1}\rangle\subset W_0$. Then the following statements hold:

1. *W decomposes orthogonally as $W \cong H \perp W_0$, where $H \cong K^2$ is a hyperbolic plane.*

2. *L_0 is self-dual in W_0.*

3. *For each i, one has*
$$y_i = \langle x, g_i \rangle.$$

4. *Consequently,*
$$L = \mathrm{Span}\Big\{(1,0,x),\ (-\langle x, g_i\rangle,\ -c\langle x, g_i\rangle,\ g_i)\ :\ 1 \le i \le n-1\Big\},$$
so that L is obtained from L_0 by a q-ary building-up construction.

5. *Conversely, two reverse statements hold:*

   (5a) *Let $L_0 \subset K^{2n-2}$ be self-dual, let $g_1, \dots, g_{n-1}$ be a basis of L_0, and let $x \in K^{2n-2}$ satisfy*
   $$\langle x, x\rangle = -1,$$
   then the row family generated by
   $$r_0 = (1,0,x), \qquad r_i = (-\langle x, g_i\rangle,\ -c\langle x, g_i\rangle,\ g_i)$$
   is linearly independent and generates a self-dual code of length $2n$.

   (5b) *More generally, if one starts from any generating family of a self-dual parent, the same formula always produces a self-orthogonal child family, and that child is self-dual as soon as its row span has dimension $n = \frac{1}{2}\dim W$.*

Proof. Since $q \equiv 1 \pmod 4$, the element -1 is a square in K, so there exists $c \in K^\times$ with $c^2 = -1$. It follows that the Euclidean plane K^2 is isometric to a hyperbolic plane. Hence there exists an orthogonal decomposition
$$W \cong H \perp W_0,$$
where $\dim H = 2$ and $\dim W_0 = 2n-2$. This proves (1).

We next prove (2). Since L is self-dual, it is self-orthogonal. Let
$$r_0 = (1,0,x), \qquad r_i = (-y_i, -cy_i, g_i).$$
For $1 \le i, j \le n-1$, we compute
$$\langle r_i, r_j\rangle = y_i y_j + c^2 y_i y_j + \langle g_i, g_j\rangle = (1+c^2) y_i y_j + \langle g_i, g_j\rangle.$$
Since $c^2 = -1$, the first term vanishes, and thus
$$\langle r_i, r_j\rangle = \langle g_i, g_j\rangle.$$

Because L is self-orthogonal, $\langle r_i, r_j\rangle = 0$, hence $\langle g_i, g_j\rangle = 0$ for all i, j. Therefore L_0 is self-orthogonal. Since $\dim L = n$ and $r_0 \notin L_0$, we have $\dim L_0 = n - 1 = \frac{1}{2}\dim W_0$. Thus L_0 is maximal totally isotropic, hence self-dual.

For (3), using self-orthogonality again,

$$0 = \langle r_0, r_i\rangle = \langle (1, 0, x), (-y_i, -cy_i, g_i)\rangle = -y_i + \langle x, g_i\rangle,$$

so $y_i = \langle x, g_i\rangle$.

Substituting this into the expression for r_i yields (4), showing that L arises from L_0 by a building-up construction.

Finally, we prove (5a) and (5b). Suppose L_0 is self-dual and $\langle x, x\rangle = -1$. Let $g_1, \ldots, g_{n-1}$ generate L_0, and define r_0, r_i as above. Then

$$\langle r_0, r_0\rangle = 1 + \langle x, x\rangle = 0,$$

$$\langle r_0, r_i\rangle = -\langle x, g_i\rangle + \langle x, g_i\rangle = 0,$$

and

$$\langle r_i, r_j\rangle = \langle g_i, g_j\rangle + (1 + c^2)\langle x, g_i\rangle\langle x, g_j\rangle = 0,$$

since L_0 is self-dual and $c^2 = -1$. Thus the row span C of these rows is always self-orthogonal.

This proves the general self-orthogonality statement in (5b). Now assume in addition that $g_1, \ldots, g_{n-1}$ is a basis of L_0. If

$$\sum_{i=1}^{n-1} a_i r_i = 0,$$

then comparing the last $2n - 2$ coordinates gives

$$\sum_{i=1}^{n-1} a_i g_i = 0.$$

Since $g_1, \ldots, g_{n-1}$ is a basis, all a_i vanish. Hence the successor rows are linearly independent. Next, r_0 does not lie in their span. Indeed, if

$$r_0 = \sum_{i=1}^{n-1} a_i r_i,$$

then the tail coordinates give

$$x = \sum_{i=1}^{n-1} a_i g_i,$$

and comparing the second coordinate yields

$$0 = \sum_{i=1}^{n-1} a_i\big(-c\langle x, g_i\rangle\big) = -c\Big\langle x, \sum_{i=1}^{n-1} a_i g_i\Big\rangle = -c\langle x, x\rangle = c,$$

a contradiction. Therefore $r_0, r_1, \ldots, r_{n-1}$ are linearly independent. Their span has dimension $n = \frac{1}{2} \dim K^{2n}$, so maximality upgrades self-orthogonality to self-duality.

This proves (5a), while the preceding orthogonality computation gives the weaker half-dimension criterion in (5b). □

Listing 2: Lean bundle for Theorem 2.9.

```
theorem buildRows_orthogonal
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    ∀ i j : Fin (m + 1), dot (buildRows x c G i) (buildRows x c G j) = 0

theorem buildRows_matrix_gram_zero
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    rowMatrix (buildRows x c G) * (rowMatrix (buildRows x c G)).transpose = 0

theorem buildRows_rowSpace_self_dual_of_finrank_half
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0)
    (heven : Even n)
    (hdim : Module.finrank K (rowSpace (buildRows x c G)) = (n + 2) / 2) :
    rowSpace (buildRows x c G) =
      (dotBilin (K := K) (n := 2 + n)).orthogonal
        (rowSpace (buildRows x c G))

theorem isSplitBuildFamily_iff_rowwise :
    IsSplitBuildFamily x c R <->
      R 0 = r0 x /\
      ∀ i : Fin m, R (Fin.succ i) =
        ri c (dot x (splitTail (R (Fin.succ i))))
          (splitTail (R (Fin.succ i)))

theorem splitBuildFamily_iff_rebuild :
    IsSplitBuildFamily x c R <->
      buildRows x c (deleteHyperbolicPairSplit R) = R

theorem exists_unique_split_parent_of_IsSplitBuildFamily
    (hR : IsSplitBuildFamily x c R) :
    ∃! G : Fin m → Fin n → K, buildRows x c G = R

theorem exists_unique_splitBuildFamily_of_parent :
    ∃! R : Fin (m + 1) → Fin (2 + n) → K,
```

```
      IsSplitBuildFamily x c R ∧ deleteHyperbolicPairSplit R = G

theorem buildRows_linearIndependent_of_linearIndependent
    (hc : c ^ 2 = (-1 : K))
    (hGlin : LinearIndependent K G) :
    LinearIndependent K (buildRows x c G)

theorem buildRows_finrank_of_linearIndependent
    (hlin : LinearIndependent K (buildRows x c G)) :
    Module.finrank K (rowSpace (buildRows x c G)) = m + 1

theorem buildRows_rowSpace_self_dual_of_self_dual_parent_basis
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0)
    (hGlin : LinearIndependent K G)
    (heven : Even n)
    (hcard : m + 1 = (n + 2) / 2) :
    rowSpace (buildRows x c G) =
      (dotBilin (K := K) (n := 2 + n)).orthogonal
        (rowSpace (buildRows x c G))
```

Remark 2.10 The stronger basis-version appearing in Theorem 2.9(5) is the forward form used in the classification section. In Lean it is formalized by the linear-independence theorem for `buildRows`, the corresponding finrank computation, and the resulting self-duality theorem for basis-indexed parent data.

Remark 2.11 (Relation with Chinburg–Zhang) The binary theorem of Chinburg–Zhang realizes a binary self-dual code as the image of an étale-cohomological map

$$\Phi : H^1_{\text{ét}}(U, \mu_2) \longrightarrow \bigoplus_{v \in S} H^1_{\text{ét}}(K_v, \mu_2),$$

where the image is Lagrangian with respect to the bilinear form induced by local cup products and the localization boundary map. The present theorem over $\mathbb{F}_q$ with $q \equiv 1 \pmod 4$ does not assert a literal μ_2-cohomological realization. Rather, it identifies the split quadratic mechanism that plays the same structural role: the existence of $c \in \mathbb{F}_q^\times$ with $c^2 = -1$, the hyperbolic decomposition of the Euclidean plane, the isotropic-line description of correction terms, and the existence of vectors of prescribed norm. In this sense, the split q-ary building-up construction is the finite-field analogue of the Chinburg–Zhang Lagrangian reduction/extension principle.

2.4 Arithmetic of $\mathbb{F}_q$ with $q \equiv 1 \pmod 4$

The condition $q \equiv 1 \pmod 4$ is equivalent to -1 being a square in $\mathbb{F}_q^\times$. We fix once and for all an element

$$c \in \mathbb{F}_q^\times \qquad \text{with} \qquad c^2 = -1.$$

This choice is used throughout the building-up formulas.

Proposition 2.12 *The existence of c has the following consequences.*

(i) The element c has multiplicative order 4.

(ii) The Euclidean plane $(\mathbb{F}_q^2, \langle \cdot, \cdot \rangle)$ is hyperbolic.

(iii) For every $k \geq 1$, the matrix equation $AA^T = -I_k$ has the explicit solution $A = cI_k$.

(iv) The binary norm form $x^2 + y^2$ splits as

$$x^2 + y^2 = (x - cy)(x + cy),$$

hence it represents every element of $\mathbb{F}_q$.

2.5 Hyperbolic Planes and Isotropic Subspaces

Definition 2.3 (Hyperbolic Plane) A **hyperbolic plane** over $\mathbb{F}_q$ is a 2-dimensional bilinear space with basis $\{e_1, e_2\}$ satisfying

$$\langle e_1, e_1 \rangle = \langle e_2, e_2 \rangle = 0, \qquad \langle e_1, e_2 \rangle = 1.$$

Any 1-dimensional totally isotropic subspace is called an **isotropic line**.

Proposition 2.13 *Let $c^2 = -1$. Then the vectors*

$$e_1 = (1, c), \qquad e_2 = (2^{-1}, -2^{-1}c)$$

form a hyperbolic basis of $\mathbb{F}_q^2$. Equivalently, the line spanned by $(1, c)$ is isotropic and the Euclidean plane is split.

Proof. We compute

$$\langle e_1, e_1 \rangle = 1 + c^2 = 0, \qquad \langle e_2, e_2 \rangle = 2^{-2}(1 + c^2) = 0,$$

and

$$\langle e_1, e_2 \rangle = 2^{-1}(1 - c^2) = 2^{-1}(1 - (-1)) = 1.$$

Thus $\{e_1, e_2\}$ is a hyperbolic basis. $\square$

2.6 Quadratic Forms over Finite Fields

Over a finite field of odd characteristic, the Witt decomposition classifies non-degenerate quadratic spaces by dimension, discriminant, and Witt index. In the present split situation, the Euclidean form

$$x_1^2 + \cdots + x_{2k}^2$$

on $\mathbb{F}_q^{2k}$ is hyperbolic for every $k \geq 1$. Consequently, $\mathbb{F}_q^{2k}$ contains maximal totally isotropic subspaces of dimension k, so self-dual codes exist in every even length. The later building-up constructions should therefore be viewed as explicit coordinatizations of this split quadratic geometry rather than as isolated matrix identities.

3 The Building-Up Construction

3.1 A Lean-verified algebraic core of the building-up construction

In this subsection, we isolate the algebraic core of the building-up construction in the split case, namely when -1 is a square. The corresponding symbolic verifications were carried out in Lean, and we record below both the mathematical statements and the successful Lean formalizations.

3.1.1 The split hyperbolic plane and its matrix realization

Theorem 3.1 (Existence of a square root of -1 and hyperbolic realization) *Let F be a finite field such that $|F| \equiv 1 \pmod 4$. Then there exists an element $i \in F$ satisfying*

$$i^2 = -1.$$

For such an i, define

$$P_i = \begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix}, \qquad H_2 := \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}.$$

Then

$$P_i^\top P_i = H_2,$$

and moreover

$$\det(P_i) \neq 0.$$

Hence the standard Euclidean plane over F is congruent to the split hyperbolic plane.

Proof. Since $|F| \equiv 1 \pmod 4$, the standard criterion for finite fields implies that -1 is a square in F. Hence there exists $i \in F$ such that

$$i^2 = -1.$$

Now consider

$$P_i = \begin{pmatrix} 1 & 1 \\ i & -i \end{pmatrix}.$$

A direct computation gives

$$P_i^\top P_i = \begin{pmatrix} 1+i^2 & 1-i^2 \\ 1-i^2 & 1+i^2 \end{pmatrix}.$$

Substituting $i^2 = -1$, we obtain

$$1 + i^2 = 1 + (-1) = 0, \qquad 1 - i^2 = 1 - (-1) = 2.$$

Therefore

$$P_i^\top P_i = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix} = H_2.$$

Next,

$$\det(P_i) = 1 \cdot (-i) - 1 \cdot i = -2i.$$

Since $i^2 = -1$, we must have $i \neq 0$. Also, because $|F| \equiv 1 \pmod 4$, the field F has odd characteristic, so $2 \neq 0$. Hence

$$-2i \neq 0,$$

and thus $\det(P_i) \neq 0$.

Therefore P_i is invertible and transforms the standard Gram matrix to the hyperbolic Gram matrix H_2. This proves that the Euclidean plane over F is split hyperbolic. □

Listing 3: Lean verification for Theorem 3.1.

```
import Mathlib

open FiniteField
open Matrix

section BuildingUpMatrixVersionDet

theorem exists_sq_neg_one_of_card_mod_four_eq_one
    {F : Type*} [Field F] [Fintype F]
    (hF : Fintype.card F % 4 = 1) :
    ∃ i : F, i ^ 2 = (-1 : F) := by
  have hneq : Fintype.card F % 4 ≠ 3 := by
    omega
  have hsq : IsSquare (-1 : F) := by
    exact (FiniteField.isSquare_neg_one_iff).2 hneq
  rcases hsq with <i, hi>
  refine <i, ?_>
  simpa [pow_two, mul_comm] using hi.symm
```

```
def H2 {K : Type*} [Field K] : Matrix (Fin 2) (Fin 2) K :=
  !![0, (2 : K); (2 : K), 0]

def P_of_root {K : Type*} [Field K] (i : K) : Matrix (Fin 2) (Fin 2) K :=
  !![1, 1; i, -i]

theorem transpose_mul_self_P_of_sq_eq_neg_one
    {K : Type*} [Field K] {i : K}
    (hi : i ^ 2 = (-1 : K)) :
    (P_of_root i).transpose * P_of_root i = H2 := by
  ext a b <;> fin_cases a <;> fin_cases b
  .
    simp [P_of_root, H2, Matrix.mul_apply]
    calc
      (1 : K) + i * i = 1 + i ^ 2 := by rw [pow_two]
      _ = 1 + (-1 : K) := by rw [hi]
      _ = 0 := by ring
  .
    simp [P_of_root, H2, Matrix.mul_apply]
    calc
      (1 : K) + -(i * i) = 1 - i ^ 2 := by rw [pow_two]; ring
      _ = 1 - (-1 : K) := by rw [hi]
      _ = (2 : K) := by ring
  .
    simp [P_of_root, H2, Matrix.mul_apply]
    calc
      (1 : K) + -(i * i) = 1 - i ^ 2 := by rw [pow_two]; ring
      _ = 1 - (-1 : K) := by rw [hi]
      _ = (2 : K) := by ring
  .
    simp [P_of_root, H2, Matrix.mul_apply]
    calc
      (1 : K) + i * i = 1 + i ^ 2 := by rw [pow_two]
      _ = 1 + (-1 : K) := by rw [hi]
      _ = 0 := by ring

theorem det_P_of_root
    {K : Type*} [Field K] (i : K) :
    Matrix.det (P_of_root i) = -(2 : K) * i := by
  simp [P_of_root, Matrix.det_fin_two]
  ring

theorem root_neg_one_ne_zero
    {K : Type*} [Field K] {i : K}
    (hi : i ^ 2 = (-1 : K)) :
    i ≠ 0 := by
  intro hiz
```

```
  have hzero : (0 : K) = (-1 : K) := by
    simpa [hiz, pow_two] using hi
  exact (neg_ne_zero.mpr one_ne_zero) hzero.symm

theorem det_P_of_root_ne_zero
    {K : Type*} [Field K] [Fact ((2 : K) ≠ 0)] {i : K}
    (hi : i ^ 2 = (-1 : K)) :
    Matrix.det (P_of_root i) ≠ 0 := by
  rw [det_P_of_root]
  have h2 : -(2 : K) ≠ 0 := by
    exact neg_ne_zero.mpr Fact.out
  exact mul_ne_zero h2 (root_neg_one_ne_zero hi)

theorem exists_nonsingular_matrix_hyperbolic_of_card_mod_four_eq_one
    {F : Type*} [Field F] [Fintype F] [Fact ((2 : F) ≠ 0)]
    (hF : Fintype.card F % 4 = 1) :
    ∃ P : Matrix (Fin 2) (Fin 2) F,
      P.transpose * P = H2 ∧ Matrix.det P ≠ 0 := by
  rcases exists_sq_neg_one_of_card_mod_four_eq_one (F := F) hF with <i, hi>
  refine <P_of_root i, ?_, ?_>
  · exact transpose_mul_self_P_of_sq_eq_neg_one (K := F) hi
  · exact det_P_of_root_ne_zero (K := F) hi

end BuildingUpMatrixVersionDet
```

3.1.2 Constructive norm witnesses and extension vectors

The survey draft aimed to make the existence input for the building-up construction completely explicit. In the current integrated development, this part is now formalized directly: once $c^2 = -1$, the split norm form $x^2 + y^2$ not only factors, but admits a closed-form witness for every target value.

Theorem 3.2 (Constructive norm witnesses from a square root of -1) *Let K be a field with $2 \neq 0$, and let $c \in K$ satisfy $c^2 = -1$. For any $a \in K$, define*

$$x_a := \frac{1+a}{2}, \qquad y_a := \frac{1-a}{2}\, c.$$

Then

$$x_a^2 + y_a^2 = a.$$

Consequently, the norm form $N(x, y) = x^2 + y^2$ is surjective onto K. Equivalently, for every $n \geq 0$ there exists a vector

$$v \in K^{n+2}$$

with

$$\langle v, v \rangle = a,$$

and in particular there exists an extension vector of norm -1 *in every ambient dimension at least* 2.

Proof. Using $c^2 = -1$, one computes

$$x_a^2 + y_a^2 = \left(\frac{1+a}{2}\right)^2 + \left(\frac{1-a}{2}c\right)^2 = \frac{(1+a)^2}{4} + \frac{(1-a)^2c^2}{4} = \frac{(1+a)^2 - (1-a)^2}{4} = a.$$

This gives an explicit witness for every target value a. Embedding (x_a, y_a) into the first two coordinates and padding the remaining coordinates with zeros yields a vector of any larger even ambient length with the same self-dot-product. □

Listing 4: Lean verification for Theorem 3.2.

```
section NormFormConstruction

variable {K : Type*} [Field K] [Fact ((2 : K) ≠ 0)]

theorem norm_form_witness
    (a c : K) (hc : c ^ 2 = (-1 : K)) :
    let x : K := (1 + a) / 2
    let y : K := ((1 - a) / 2) * c
    x ^ 2 + y ^ 2 = a := by
  dsimp
  have h2 : (2 : K) ≠ 0 := Fact.out
  field_simp [h2]
  rw [hc]
  ring

theorem exists_norm_pair
    (a c : K) (hc : c ^ 2 = (-1 : K)) :
    ∃ x y : K, x ^ 2 + y ^ 2 = a := by
  refine <(1 + a) / 2, ((1 - a) / 2) * c, ?_>
  simpa using norm_form_witness (K := K) a c hc

theorem exists_dot_eq_in_two_coordinates
    {n : ℕ} (a c : K) (hc : c ^ 2 = (-1 : K)) :
    ∃ v : Fin (2 + n) → K, dot v v = a := by
  rcases exists_norm_pair (K := K) a c hc with <x, y, hxy>
  refine <prepend2 x y (0 : Fin n → K), ?_>
  rw [dot_prepend2_prepend2]
  simp [dot]
  simpa [pow_two] using hxy

theorem exists_extension_vector_neg_one
    {n : ℕ} (c : K) (hc : c ^ 2 = (-1 : K)) :
    ∃ v : Fin (2 + n) → K, dot v v = (-1 : K) := by
```

```
  simpa using exists_dot_eq_in_two_coordinates (K := K) (n := n) (-1 : K) c hc

end NormFormConstruction
```

3.1.3 Hyperbolic pairs and standard seed rows

To prepare the eventual completeness and induction arguments, it is useful to isolate two more minimal ingredients. First, the isotropic line generated by $(1, c)$ comes with its opposite partner $(1, -c)$, forming a hyperbolic pair. Second, the standard self-dual seed rows

$$s_i = (e_i,\ ce_i) \in K^{n+n}, \qquad i = 1, \ldots, n,$$

already form a pairwise orthogonal row family as soon as $c^2 = -1$. In fact, their row span already gives the canonical split self-dual base code of length $2n$. These are the smallest reusable blocks for building base codes in the split case.

Proposition 3.3 (Hyperbolic pair from a square root of -1) *Let K be a field, let $c \in K$ satisfy $c^2 = -1$, and write*

$$e_+ = (1, c, 0, \ldots, 0), \qquad e_- = (1, -c, 0, \ldots, 0) \in K^{2+n}.$$

Then both e_+ and e_- are isotropic, and

$$\langle e_+, e_- \rangle = 2.$$

Thus the two vectors form a hyperbolic pair in the split plane.

Proof. The self-pairings are

$$\langle e_+, e_+ \rangle = 1 + c^2 = 0, \qquad \langle e_-, e_- \rangle = 1 + (-c)^2 = 1 + c^2 = 0.$$

For the mixed pairing,

$$\langle e_+, e_- \rangle = 1 + c(-c) = 1 - c^2 = 1 - (-1) = 2.$$

□

Proposition 3.4 (Standard split seed rows) *Let K be a field and let $c \in K$ satisfy $c^2 = -1$. For each $i \in \{1, \ldots, n\}$, let*

$$s_i = (e_i,\ ce_i) \in K^{n+n},$$

where e_i is the i-th standard basis vector of K^n. Then

$$\langle s_i, s_j \rangle = 0 \qquad \text{for all } i, j.$$

Equivalently, the matrix with rows $s_1, \ldots, s_n$ has Gram matrix zero, and its row span is self-orthogonal.

Proof. If $i \neq j$, then the supports are disjoint in both halves, so $\langle s_i, s_j \rangle = 0$. If $i = j$, then

$$\langle s_i, s_i \rangle = 1 + c^2 = 0.$$

Hence the whole family is pairwise orthogonal. The Gram-matrix and row-span statements follow from bilinearity exactly as in the building-up family. □

Listing 5: Lean verification for Propositions 3.3 and 3.4.

```
def IsIsotropic {n : ℕ} (v : Fin n → K) : Prop :=
  dot v v = 0

def PairwiseOrthogonal {m n : ℕ} (R : Fin m → Fin n → K) : Prop :=
  ∀ i j : Fin m, dot (R i) (R j) = 0

def hyperbolicVecPlus {n : ℕ} (c : K) : Fin (2 + n) → K :=
  prepend2 1 c 0

def hyperbolicVecMinus {n : ℕ} (c : K) : Fin (2 + n) → K :=
  prepend2 1 (-c) 0

theorem hyperbolicVecPlus_isotropic
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    IsIsotropic (K := K) (hyperbolicVecPlus (n := n) c) := ...

theorem hyperbolicVecMinus_isotropic
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    IsIsotropic (K := K) (hyperbolicVecMinus (n := n) c) := ...

theorem dot_hyperbolicVecPlus_hyperbolicVecMinus
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    dot (hyperbolicVecPlus (K := K) (n := n) c)
        (hyperbolicVecMinus (K := K) (n := n) c) = (2 : K) := ...

def seedRow {n : ℕ} (c : K) (i : Fin n) : Fin (n + n) → K :=
  Fin.append (Pi.single i 1) (Pi.single i c)

def seedRows {n : ℕ} (c : K) : Fin n → Fin (n + n) → K :=
  fun i => seedRow c i

theorem seedRows_orthogonal
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    PairwiseOrthogonal (K := K) (seedRows (K := K) (n := n) c) := ...

theorem seedRows_matrix_gram_zero
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    rowMatrix (seedRows (K := K) (n := n) c) *
      (rowMatrix (seedRows (K := K) (n := n) c)).transpose = 0 := ...
```

Theorem 3.5 (Standard split seed code is self-dual) *Let K be a field and let $c \in K$ satisfy $c^2 = -1$. Then the row span of the seed rows*

$$s_i = (e_i,\ ce_i) \in K^{n+n}$$

is equal to its Euclidean orthogonal complement in K^{n+n}.

Proof. The inclusion $C \subseteq C^\perp$ follows from Proposition 3.4. For the reverse inclusion, write any vector $v \in K^{n+n}$ as $v = (a, b)$ with left and right halves $a, b \in K^n$. Orthogonality to every seed row $s_i = (e_i, ce_i)$ implies

$$0 = \langle v, s_i \rangle = a_i + cb_i \qquad \text{for all } i,$$

and multiplying by c gives $b_i = ca_i$ because $c^2 = -1$. Hence $v = (a, ca)$, so v lies in the span of the seed rows. Therefore $C^\perp \subseteq C$, and thus $C = C^\perp$. □

Listing 6: Lean verification for Theorem 3.5.

```
def seedEmbedding {n : ℕ} (c : K) (u : Fin n → K) : Fin (n + n) → K :=
  Fin.append u (c • u)

theorem seedEmbedding_eq_sum_seedRows
    {n : ℕ} (c : K) (u : Fin n → K) :
    seedEmbedding (K := K) c u =
      ∑ i, u i • seedRows (K := K) (n := n) c i := ...

theorem seedEmbedding_mem_rowSpace
    {n : ℕ} (c : K) (u : Fin n → K) :
    seedEmbedding (K := K) c u ∈
      rowSpace (seedRows (K := K) (n := n) c) := ...

theorem seedRows_rowSpace_eq_orthogonal
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    rowSpace (seedRows (K := K) (n := n) c) =
      (dotBilin (K := K) (n := n + n)).orthogonal
        (rowSpace (seedRows (K := K) (n := n) c)) := ...

theorem split_seed_code_self_dual
    {n : ℕ} {c : K} (hc : c ^ 2 = (-1 : K)) :
    rowSpace (seedRows (K := K) (n := n) c) =
      (dotBilin (K := K) (n := n + n)).orthogonal
        (rowSpace (seedRows (K := K) (n := n) c)) := ...
```

3.1.4 The building-up rows

Definition 3.1 (Dot product and building-up rows [8]) *Let K be a field. For vectors $u = (u_1, \ldots, u_n)$ and $v = (v_1, \ldots, v_n)$ in K^n, define*

$$\langle u, v \rangle := \sum_{j=1}^{n} u_j v_j.$$

Let $x \in K^n$, let $c \in K$, and let

$$G = \{G_1, \ldots, G_m\} \subset K^n$$

be a family of row vectors. For each i, define

$$y_i := \langle x, G_i \rangle.$$

The associated building-up rows in K^{n+2} are

$$r_0 := (1, 0, x), \qquad r_i := (-y_i,\, -cy_i,\, G_i) \qquad (1 \le i \le m).$$

Lemma 3.6 (Decomposition of the dot product after adjoining two coordinates) *For any $a, b, a', b' \in K$ and any $u, v \in K^n$,*

$$\langle (a, b, u), (a', b', v) \rangle = aa' + bb' + \langle u, v \rangle.$$

Proof. This follows immediately by splitting the coordinate sum into the first two coordinates and the remaining n coordinates. Indeed,

$$\langle (a, b, u), (a', b', v) \rangle = aa' + bb' + \sum_{j=1}^{n} u_j v_j = aa' + bb' + \langle u, v \rangle.$$

□

Lemma 3.7 (Orthogonality relations among the building-up rows) *Assume that*

$$\langle x, x \rangle = -1, \qquad c^2 = -1, \qquad \langle G_i, G_j \rangle = 0 \quad \textit{for all } i, j.$$

Then the building-up rows satisfy:

$$\begin{aligned} &\langle r_0, r_0 \rangle = 0, \\ &\langle r_0, r_i \rangle = 0 \quad (1 \le i \le m), \\ &\langle r_i, r_j \rangle = 0 \quad (1 \le i, j \le m). \end{aligned}$$

Proof. By Lemma 3.6,

$$\langle r_0, r_0 \rangle = 1^2 + 0^2 + \langle x, x \rangle = 1 + (-1) = 0.$$

Next, using $y_i = \langle x, G_i \rangle$,

$$\langle r_0, r_i \rangle = 1 \cdot (-y_i) + 0 \cdot (-cy_i) + \langle x, G_i \rangle = -y_i + y_i = 0.$$

Finally,

$$\langle r_i, r_j \rangle = (-y_i)(-y_j) + (-cy_i)(-cy_j) + \langle G_i, G_j \rangle.$$

Since $\langle G_i, G_j \rangle = 0$, this becomes

$$\langle r_i, r_j \rangle = y_i y_j + c^2 y_i y_j.$$

Using $c^2 = -1$, we obtain

$$\langle r_i, r_j \rangle = y_i y_j + (-1) y_i y_j = 0.$$

This proves all three identities. □

Listing 7: Lean verification for Lemma 3.7.

```
import Mathlib

open FiniteField
open Matrix
open BigOperators

section BuildingUpRows

variable {K : Type*} [Field K]

def dot {n : ℕ} (u v : Fin n → K) : K :=
  ∑ j, u j * v j

def head2 (a b : K) : Fin 2 → K
  | <0, _> => a
  | <1, _> => b

def prepend2 {n : ℕ} (a b : K) (u : Fin n → K) : Fin (2 + n) → K :=
  Fin.append (head2 a b) u

def r0 {n : ℕ} (x : Fin n → K) : Fin (2 + n) → K :=
  prepend2 1 0 x

def ri {n : ℕ} (c yi : K) (g : Fin n → K) : Fin (2 + n) → K :=
  prepend2 (-yi) (-(c * yi)) g

theorem dot_prepend2_prepend2
    {n : ℕ} (a b a' b' : K) (u v : Fin n → K) :
    dot (prepend2 a b u) (prepend2 a' b' v)
      = a * a' + b * b' + dot u v := by
```

```
  unfold dot prepend2
  rw [Fin.sum_univ_add]
  simp [head2, Fin.sum_univ_two, add_assoc]

theorem dot_r0_r0
    {n : ℕ} {x : Fin n → K}
    (hx : dot x x = (-1 : K)) :
    dot (r0 x) (r0 x) = 0 := by
  rw [r0, dot_prepend2_prepend2]
  rw [hx]
  ring

theorem dot_r0_ri
    {n : ℕ} {x g : Fin n → K} {c yi : K}
    (hyi : yi = dot x g) :
    dot (r0 x) (ri c yi g) = 0 := by
  rw [r0, ri, dot_prepend2_prepend2]
  rw [hyi]
  ring

theorem dot_ri_ri
    {n : ℕ} {g h : Fin n → K} {c yi yj : K}
    (hc : c ^ 2 = (-1 : K))
    (hgh : dot g h = 0) :
    dot (ri c yi g) (ri c yj h) = 0 := by
  rw [ri, ri, dot_prepend2_prepend2]
  rw [hgh]
  calc
    (-yi) * (-yj) + (-(c * yi)) * (-(c * yj)) + 0
        = yi * yj + (c * yi) * (c * yj) := by ring
    _ = yi * yj + (c ^ 2) * (yi * yj) := by
        rw [pow_two]
        ring
    _ = yi * yj + (-1 : K) * (yi * yj) := by rw [hc]
    _ = 0 := by ring

end BuildingUpRows
```

3.1.5 From rowwise orthogonality to a self-orthogonal generator matrix

Definition 3.2 (The building-up row family) *Let $x \in K^n$ and $c \in K$. Let $G \subset K^n$ be the family $\{G_1, \ldots, G_m\}$ introduced above. Define the row family*

$$R = \{R_0, R_1, \ldots, R_m\} \subset K^{n+2}$$

by

$$R_0 := r_0, \qquad R_i := r_i \quad (1 \le i \le m).$$

Equivalently, R is obtained by taking the distinguished row $(1, 0, x)$ and adjoining the corrected rows $(-y_i, -cy_i, G_i)$.

Theorem 3.8 (Pairwise orthogonality of the building-up family) *Assume that*

$$\langle x, x\rangle = -1, \qquad c^2 = -1, \qquad \langle G_i, G_j\rangle = 0 \quad \textit{for all } i, j.$$

Then

$$\langle R_p, R_q\rangle = 0 \qquad \textit{for all } 0 \le p, q \le m.$$

Proof. By Lemma 3.7, we already know that

$$\langle r_0, r_0\rangle = 0, \qquad \langle r_0, r_i\rangle = 0, \qquad \langle r_i, r_j\rangle = 0$$

for all i, j. Since the family R consists exactly of r_0 together with the r_i, all pairwise inner products among the R_p vanish. □

Listing 8: Lean verification for Theorem 3.8.

```
import Mathlib

open FiniteField
open Matrix
open BigOperators

section BuildingUpFamily

variable {K : Type*} [Field K]

def dot {n : ℕ} (u v : Fin n → K) : K :=
 ∑ j, u j * v j

theorem dot_comm {n : ℕ} (u v : Fin n → K) :
 dot u v = dot v u := by
 unfold dot
 apply Finset.sum_congr rfl
 intro j hj
 ring

def head2 (a b : K) : Fin 2 → K
| <0, _> => a
| <1, _> => b

def prepend2 {n : ℕ} (a b : K) (u : Fin n → K) : Fin (2 + n) → K :=
 Fin.append (head2 a b) u

def r0 {n : ℕ} (x : Fin n → K) : Fin (2 + n) → K :=
```

```
 prepend2 1 0 x

def ri {n : ℕ} (c yi : K) (g : Fin n → K) : Fin (2 + n) → K :=
 prepend2 (-yi) (-(c * yi)) g

theorem dot_prepend2_prepend2
 {n : ℕ} (a b a' b' : K) (u v : Fin n → K) :
 dot (prepend2 a b u) (prepend2 a' b' v)
 = a * a' + b * b' + dot u v := by
 unfold dot prepend2
 rw [Fin.sum_univ_add]
 simp [head2, Fin.sum_univ_two, add_assoc]

theorem dot_r0_r0
 {n : ℕ} {x : Fin n → K}
 (hx : dot x x = (-1 : K)) :
 dot (r0 x) (r0 x) = 0 := by
 rw [r0, dot_prepend2_prepend2]
 rw [hx]
 ring

theorem dot_r0_ri
 {n : ℕ} {x g : Fin n → K} {c yi : K}
 (hyi : yi = dot x g) :
 dot (r0 x) (ri c yi g) = 0 := by
 rw [r0, ri, dot_prepend2_prepend2]
 rw [hyi]
 ring

theorem dot_ri_ri
 {n : ℕ} {g h : Fin n → K} {c yi yj : K}
 (hc : c ^ 2 = (-1 : K))
 (hgh : dot g h = 0) :
 dot (ri c yi g) (ri c yj h) = 0 := by
 rw [ri, ri, dot_prepend2_prepend2]
 rw [hgh]
 calc
 (-yi) * (-yj) + (-(c * yi)) * (-(c * yj)) + 0
 = yi * yj + (c * yi) * (c * yj) := by ring
 _ = yi * yj + (c ^ 2) * (yi * yj) := by
 rw [pow_two]
 ring
 _ = yi * yj + (-1 : K) * (yi * yj) := by rw [hc]
 _ = 0 := by ring

def buildRows {m n : ℕ} (x : Fin n → K) (c : K) (G : Fin m → Fin n → K) :
 Fin (m + 1) → Fin (2 + n) → K :=
```

```
 Fin.cases (r0 x) (fun i => ri c (dot x (G i)) (G i))

theorem buildRows_zero_zero
 {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
 (hx : dot x x = (-1 : K)) :
 dot (buildRows x c G 0) (buildRows x c G 0) = 0 := by
 simp [buildRows]
 exact dot_r0_r0 hx

theorem buildRows_zero_succ
 {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K} (i : Fin m) :
 dot (buildRows x c G 0) (buildRows x c G (Fin.succ i)) = 0 := by
 simp [buildRows]
 exact dot_r0_ri rfl

theorem buildRows_succ_zero
 {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K} (i : Fin m) :
 dot (buildRows x c G (Fin.succ i)) (buildRows x c G 0) = 0 := by
 rw [dot_comm]
 exact buildRows_zero_succ i

theorem buildRows_succ_succ
 {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
 (hc : c ^ 2 = (-1 : K))
 (hG : ∀ i j : Fin m, dot (G i) (G j) = 0)
 (i j : Fin m) :
 dot (buildRows x c G (Fin.succ i)) (buildRows x c G (Fin.succ j)) = 0 := by
 simp [buildRows]
 exact dot_ri_ri (hc := hc) (hgh := hG i j)

theorem buildRows_orthogonal
 {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
 (hx : dot x x = (-1 : K))
 (hc : c ^ 2 = (-1 : K))
 (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
 ∀ i j : Fin (m + 1), dot (buildRows x c G i) (buildRows x c G j) = 0 := by
 intro i j
rcases Fin.eq_zero_or_eq_succ i with rfl | <i, rfl>
· rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
 · exact buildRows_zero_zero hx
 · exact buildRows_zero_succ j
· rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
 · exact buildRows_succ_zero i
 · exact buildRows_succ_succ (hc := hc) (hG := hG) i j

end BuildingUpFamily
```

Definition 3.3 (The building-up matrix) *Let*

$$A = \begin{pmatrix} R_0 \\ R_1 \\ \vdots \\ R_m \end{pmatrix} \in M_{m+1,n+2}(K)$$

be the matrix whose rows are the building-up family.

Lemma 3.9 (Entries of the Gram matrix) *Let A be any matrix over K with rows $R_0, \ldots, R_m$. Then the (p, q)-entry of $AA^\top$ is*

$$(AA^\top)_{pq} = \langle R_p, R_q \rangle.$$

Proof. By matrix multiplication,

$$(AA^\top)_{pq} = \sum_k A_{pk}(A^\top)_{kq} = \sum_k A_{pk}A_{qk}.$$

But this is precisely the dot product of the p-th and q-th rows of A. □

Theorem 3.10 (Vanishing of the Gram matrix) *Assume that*

$$\langle x, x \rangle = -1, \qquad c^2 = -1, \qquad \langle G_i, G_j \rangle = 0 \quad \textit{for all } i, j.$$

Let A be the building-up matrix. Then

$$AA^\top = 0.$$

Proof. By Lemma 3.9, every entry of $AA^\top$ is the dot product of two rows of A. By Theorem 3.8, every such dot product is zero. Hence each entry of $AA^\top$ vanishes, so

$$AA^\top = 0.$$

□

Listing 9: Lean verification for Theorem 3.10.

```
import Mathlib

open FiniteField
open Matrix
open BigOperators

section BuildingUpMatrixGram

variable {K : Type*} [Field K]
```

```
def dot {n : ℕ} (u v : Fin n → K) : K :=
  ∑ j, u j * v j

theorem dot_comm {n : ℕ} (u v : Fin n → K) :
    dot u v = dot v u := by
  unfold dot
  apply Finset.sum_congr rfl
  intro j hj
  ring

def head2 (a b : K) : Fin 2 → K
  | <0, _> => a
  | <1, _> => b

def prepend2 {n : ℕ} (a b : K) (u : Fin n → K) : Fin (2 + n) → K :=
  Fin.append (head2 a b) u

def r0 {n : ℕ} (x : Fin n → K) : Fin (2 + n) → K :=
  prepend2 1 0 x

def ri {n : ℕ} (c yi : K) (g : Fin n → K) : Fin (2 + n) → K :=
  prepend2 (-yi) (-(c * yi)) g

theorem dot_prepend2_prepend2
    {n : ℕ} (a b a' b' : K) (u v : Fin n → K) :
    dot (prepend2 a b u) (prepend2 a' b' v)
      = a * a' + b * b' + dot u v := by
  unfold dot prepend2
  rw [Fin.sum_univ_add]
  simp [head2, Fin.sum_univ_two, add_assoc]

theorem dot_r0_r0
    {n : ℕ} {x : Fin n → K}
    (hx : dot x x = (-1 : K)) :
    dot (r0 x) (r0 x) = 0 := by
  rw [r0, dot_prepend2_prepend2]
  rw [hx]
  ring

theorem dot_r0_ri
    {n : ℕ} {x g : Fin n → K} {c yi : K}
    (hyi : yi = dot x g) :
    dot (r0 x) (ri c yi g) = 0 := by
  rw [r0, ri, dot_prepend2_prepend2]
  rw [hyi]
  ring
```

```
theorem dot_ri_ri
    {n : ℕ} {g h : Fin n → K} {c yi yj : K}
    (hc : c ^ 2 = (-1 : K))
    (hgh : dot g h = 0) :
    dot (ri c yi g) (ri c yj h) = 0 := by
  rw [ri, ri, dot_prepend2_prepend2]
  rw [hgh]
  calc
    (-yi) * (-yj) + (-(c * yi)) * (-(c * yj)) + 0
        = yi * yj + (c * yi) * (c * yj) := by ring
    _ = yi * yj + (c ^ 2) * (yi * yj) := by
        rw [pow_two]
        ring
    _ = yi * yj + (-1 : K) * (yi * yj) := by rw [hc]
    _ = 0 := by ring

def buildRows {m n : ℕ} (x : Fin n → K) (c : K) (G : Fin m → Fin n → K) :
    Fin (m + 1) → Fin (2 + n) → K :=
  Fin.cases (r0 x) (fun i => ri c (dot x (G i)) (G i))

theorem buildRows_zero_zero
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
    (hx : dot x x = (-1 : K)) :
    dot (buildRows x c G 0) (buildRows x c G 0) = 0 := by
  simp [buildRows]
  exact dot_r0_r0 hx

theorem buildRows_zero_succ
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K} (i : Fin m) :
    dot (buildRows x c G 0) (buildRows x c G (Fin.succ i)) = 0 := by
  simp [buildRows]
  exact dot_r0_ri rfl

theorem buildRows_succ_zero
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K} (i : Fin m) :
    dot (buildRows x c G (Fin.succ i)) (buildRows x c G 0) = 0 := by
  rw [dot_comm]
  exact buildRows_zero_succ i

theorem buildRows_succ_succ
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0)
    (i j : Fin m) :
    dot (buildRows x c G (Fin.succ i)) (buildRows x c G (Fin.succ j)) = 0 := by
  simp [buildRows]
```

```
  exact dot_ri_ri (hc := hc) (hgh := hG i j)

theorem buildRows_orthogonal
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    ∀ i j : Fin (m + 1), dot (buildRows x c G i) (buildRows x c G j) = 0 := by
  intro i j
  rcases Fin.eq_zero_or_eq_succ i with rfl | <i, rfl>
  · rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
    · exact buildRows_zero_zero hx
    · exact buildRows_zero_succ j
  · rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
    · exact buildRows_succ_zero i
    · exact buildRows_succ_succ (hc := hc) (hG := hG) i j

def rowMatrix {m n : ℕ} (R : Fin m → Fin n → K) : Matrix (Fin m) (Fin n) K :=
  fun i j => R i j

theorem rowMatrix_mul_transpose_apply
    {m n : ℕ} (R : Fin m → Fin n → K) (i j : Fin m) :
    (rowMatrix R * (rowMatrix R).transpose) i j = dot (R i) (R j) := by
  unfold rowMatrix dot
  rw [Matrix.mul_apply]
  simp

theorem buildRows_matrix_gram_zero
    {m n : ℕ} {x : Fin n → K} {c : K} {G : Fin m → Fin n → K}
    (hx : dot x x = (-1 : K))
    (hc : c ^ 2 = (-1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    rowMatrix (buildRows x c G) * (rowMatrix (buildRows x c G)).transpose = 0 := by
  ext i j
  rw [rowMatrix_mul_transpose_apply]
  exact buildRows_orthogonal (hx := hx) (hc := hc) (hG := hG) i j

end BuildingUpMatrixGram
```

Corollary 3.11 (Self-orthogonality of the row span) *Under the hypotheses of Theorem 3.10, the row span of the building-up matrix A is a self-orthogonal linear code in K^{n+2}.*

Proof. Since $AA^\top = 0$, every pair of rows of A is orthogonal. By bilinearity of the dot product, any two linear combinations of the rows are again orthogonal. Hence the row span of A is a self-orthogonal subspace of K^{n+2}. □

Corollary 3.12 (Self-duality in the maximal-dimension case) *Assume the hypotheses of Theorem 3.10, and suppose in addition that the row span of the building-up matrix A has dimension*

$$\frac{n+2}{2}.$$

Then the row span of A is self-dual.

Proof. By Corollary 3.11, the row span C of A satisfies

$$C \subseteq C^{\perp}.$$

Taking dimensions in the ambient space K^{n+2} yields

$$\dim C + \dim C^{\perp} = n + 2.$$

If $\dim C = \frac{n+2}{2}$, then necessarily

$$\dim C^{\perp} = \frac{n+2}{2} = \dim C.$$

Since $C \subseteq C^{\perp}$ and both spaces have the same dimension, we conclude that

$$C = C^{\perp}.$$

Thus C is self-dual. □

Proposition 3.13 (Normalization of a nonzero isotropic head) *Let*

$$v = (a, b, g) \in K^{n+2}$$

with $a \neq 0$ and

$$a^2 + b^2 = 0.$$

Then, setting

$$c = \frac{b}{a}, \qquad g' = a^{-1} g,$$

one has

$$c^2 = -1$$

and

$$v = a\,(1, c, g').$$

Equivalently, every row whose first two coordinates form a nonzero isotropic vector is obtained from a row in normalized split-head form by multiplication by a nonzero scalar.

Proof. Since $a \neq 0$, the quotient $c = b/a$ is defined. Dividing the identity $a^2 + b^2 = 0$ by a^2 gives

$$1 + \left(\frac{b}{a}\right)^2 = 0,$$

hence $c^2 = -1$. Writing $g = ag'$ with $g' = a^{-1}g$, we obtain

$$(a, b, g) = \left(a,\ a\frac{b}{a},\ a(a^{-1}g)\right) = a\,(1, c, g').$$

Thus the head is normalized after rescaling by the nonzero scalar a^{-1}. □

Corollary 3.14 (Orthogonal normalization relative to the distinguished row) *Let*

$$R_0 = (1, 0, x) \in K^{n+2}, \qquad v = (a, b, g) \in K^{n+2},$$

with $a \neq 0$, $a^2 + b^2 = 0$, *and*

$$\langle R_0, v\rangle = 0.$$

Then there exists $c \in K$ *and a normalized tail vector*

$$g' = a^{-1}g$$

such that

$$c^2 = -1, \qquad v = a\,(1, c, g'), \qquad \langle x, g'\rangle = -1.$$

Proof. By Proposition 3.13, taking $c = b/a$ gives $c^2 = -1$ and

$$v = a\,(1, c, g') \qquad \text{with} \qquad g' = a^{-1}g.$$

Now

$$0 = \langle R_0, v\rangle = \langle (1, 0, x),\ a(1, c, g')\rangle = a\big(1 + \langle x, g'\rangle\big).$$

Since $a \neq 0$, we conclude that

$$\langle x, g'\rangle = -1.$$

□

Corollary 3.15 (Canonical split-row form after normalization) *Under the hypotheses of Corollary 3.14, the row* v *can be written as*

$$v = a\, r_i(c, g')$$

where

$$r_i(c, g') = \big(-\langle x, g'\rangle,\ -c\langle x, g'\rangle,\ g'\big) = (1, c, g')$$

because $\langle x, g'\rangle = -1$.

Proof. The identity

$$v = a\,(1, c, g')$$

from Corollary 3.14 gives

$$v = a\, r_i(c, g')$$

once one substitutes $\langle x, g'\rangle = -1$ into the defining formula

$$r_i(c, g') = \big(-\langle x, g'\rangle,\ -c\langle x, g'\rangle,\ g'\big).$$

□

Corollary 3.16 (Familywise scaled canonical split form) *Fix $x \in K^n$ and $c \in K$ with $c^2 = -1$, and let*

$$R = \{R_0, R_1, \ldots, R_m\} \subset K^{n+2}$$

be a row family with

$$R_0 = (1, 0, x).$$

Assume that for each $i \geq 1$ the successor row has the form

$$R_i = (a_i, ca_i, g_i)$$

with $a_i \neq 0$, and is orthogonal to R_0. Then for each $i \geq 1$ there exists a normalized tail vector $g_i' \in K^n$ such that

$$\langle x, g_i' \rangle = -1 \qquad \text{and} \qquad R_i = a_i\, r_i(c, g_i').$$

Proof. Apply Corollary 3.15 to each successor row separately. The hypothesis

$$R_i = (a_i, ca_i, g_i)$$

already puts the head on the fixed c-isotropic line. The orthogonality condition

$$\langle R_0, R_i \rangle = 0$$

forces the normalized tail to satisfy $\langle x, g_i' \rangle = -1$. Hence every successor row is a nonzero scalar multiple of the canonical split row determined by its normalized tail. □

Proposition 3.17 (Rowwise nonzero scaling preserves the generated code) *Let*

$$R = \{R_1, \ldots, R_m\} \subset K^n$$

be a row family, and let $\lambda_1, \ldots, \lambda_m \in K^\times$. Define a new family $R' = \{R_1', \ldots, R_m'\}$ by

$$R_i' = \lambda_i R_i \qquad (1 \leq i \leq m).$$

Then the row spans of R and R' are equal.

Proof. Each row R_i' belongs to the span of R, so the row span of R' is contained in the row span of R. Conversely,

$$R_i = \lambda_i^{-1} R_i'$$

for every i, since each λ_i is nonzero. Hence each row of R belongs to the row span of R', and the reverse inclusion follows. □

Proposition 3.18 (Row permutation preserves the generated code) *Let*

$$R = \{R_1, \ldots, R_m\} \subset K^n$$

be a row family, and let $\sigma \in S_m$. Define

$$R_i' = R_{\sigma(i)} \qquad (1 \leq i \leq m).$$

Then the row spans of R and $R' = \{R_1', \ldots, R_m'\}$ are equal.

Proof. Each row of R' is one of the rows of R, so the row span of R' is contained in the row span of R. Conversely, since σ is a permutation, every row R_i occurs as some row $R'_{\sigma^{-1}(i)}$ of the reordered family. Hence every row of R belongs to the row span of R', and the reverse inclusion follows. □

Proposition 3.19 (Pivot normalization by row operations) *Let*

$$R = \{R_0, \dots, R_m\} \subset K^{n+2},$$

and choose an index j such that

$$R_j = (a, ca, g)$$

with $a \neq 0$. Form a new family R' by first permuting rows so that R_j becomes the first row, and then multiplying this first row by a^{-1} while leaving all other rows unchanged. Then:

1. *the row spans of R and R' are equal;*
2. *the first row of R' is*

$$(1, c, a^{-1}g).$$

Proof. The equality of row spans follows by combining Propositions 3.17 and 3.18. For the first row, permutation places R_j in the initial position, and multiplying by a^{-1} yields

$$a^{-1}(a, ca, g) = (1, c, a^{-1}g).$$

□

Proposition 3.20 (Orthogonality to a normalized isotropic pivot) *Fix $c \in K$ with $c^2 = -1$, and let*

$$u = (1, c, x), \qquad v = (a, ca, g)$$

be two rows in K^{n+2}. Then

$$\langle u, v\rangle = \langle x, g\rangle.$$

In particular,

$$\langle u, v\rangle = 0 \qquad \Longleftrightarrow \qquad \langle x, g\rangle = 0.$$

Proof. Expanding the dot product gives

$$\langle u, v\rangle = a + c(ca) + \langle x, g\rangle = a(1 + c^2) + \langle x, g\rangle.$$

Since $c^2 = -1$, the first term vanishes, leaving

$$\langle u, v\rangle = \langle x, g\rangle.$$

The equivalence follows immediately. □

Proposition 3.21 (Tail orthogonality after pivot normalization) *Fix $c \in K$ with $c^2 = -1$, and let*

$$R = \{R_0, R_1, \dots, R_m\} \subset K^{n+2}$$

be a row family with

$$R_0 = (1, c, x), \qquad R_i = (a_i, ca_i, g_i) \quad (1 \le i \le m).$$

Assume that the rows of R are pairwise orthogonal. Then:

1. *for every $i \ge 1$, one has*
$$\langle x, g_i \rangle = 0;$$
2. *for every $i, j \ge 1$, one has*
$$\langle g_i, g_j \rangle = 0.$$

Equivalently, deleting the first hyperbolic coordinate pair from the successor rows produces a pairwise orthogonal tail family in K^n, each of whose rows is orthogonal to the pivot tail x.

Proof. Applying Proposition 3.20 to the pair (R_0, R_i) gives

$$\langle R_0, R_i \rangle = \langle x, g_i \rangle.$$

Since the left-hand side vanishes by hypothesis, $\langle x, g_i \rangle = 0$ for every $i \ge 1$. Likewise, for $i, j \ge 1$,

$$\langle R_i, R_j \rangle = a_i a_j (1 + c^2) + \langle g_i, g_j \rangle = \langle g_i, g_j \rangle,$$

because $c^2 = -1$. Hence pairwise orthogonality of the rows of R implies pairwise orthogonality of the tail family. □

Corollary 3.22 (Orthogonal tail-family package after pivot normalization) *Under the hypotheses of Proposition 3.21, the tail family*

$$G = \{g_1, \dots, g_m\} \subset K^n$$

satisfies

$$\langle x, g_i \rangle = 0 \qquad (1 \le i \le m)$$

and is pairwise orthogonal.

Proof. This is exactly the pair of conclusions established in Proposition 3.21, rewritten in terms of the deleted-tail family G. □

Theorem 3.23 (Rowwise characterization of pivot-normalized orthogonal families) *Fix $c \in K$ with $c^2 = -1$, a pivot tail $x \in K^n$, nonzero-index scalars $a_1, \dots, a_m \in K$, and tails $g_1, \dots, g_m \in K^n$. Define*

$$R_0 = (1, c, x), \qquad R_i = (a_i, ca_i, g_i) \quad (1 \le i \le m).$$

Then the row family $R = \{R_0, R_1, \dots, R_m\}$ is pairwise orthogonal if and only if

$$\langle x, x \rangle = 0, \qquad \langle x, g_i \rangle = 0 \text{ for all } i, \qquad \langle g_i, g_j \rangle = 0 \text{ for all } i, j.$$

Proof. If R is pairwise orthogonal, then applying orthogonality to (R_0, R_0) gives

$$0 = \langle R_0, R_0 \rangle = 1 + c^2 + \langle x, x \rangle = \langle x, x \rangle,$$

because $c^2 = -1$. Applying Proposition 3.21 then yields

$$\langle x, g_i \rangle = 0 \qquad \text{and} \qquad \langle g_i, g_j \rangle = 0$$

for all relevant indices.

Conversely, assume

$$\langle x, x \rangle = 0, \qquad \langle x, g_i \rangle = 0, \qquad \langle g_i, g_j \rangle = 0.$$

Then

$$\langle R_0, R_0 \rangle = 1 + c^2 + \langle x, x \rangle = 0.$$

For each $i \geq 1$, Proposition 3.20 gives

$$\langle R_0, R_i \rangle = \langle x, g_i \rangle = 0.$$

For $i, j \geq 1$, the same expansion as in Proposition 3.21 yields

$$\langle R_i, R_j \rangle = a_i a_j (1 + c^2) + \langle g_i, g_j \rangle = 0.$$

Hence all rows are pairwise orthogonal. □

Theorem 3.24 (Pivot normalization produces a pivot-tail package) *Fix $c \in K$ with $c^2 = -1$, and let*

$$R = \{R_0, \ldots, R_m\} \subset K^{n+2}$$

be a pairwise orthogonal row family. Choose an index j and assume

$$R_j = (a, ca, g) \qquad \text{with} \qquad a \neq 0.$$

Assume further that every row of R lies on the same c-isotropic line, that is, for each k there exist $\lambda_k \in K$ and $h_k \in K^n$ such that

$$R_k = (\lambda_k, c\lambda_k, h_k).$$

Let R' be obtained from R by moving the pivot row R_j to the first position and multiplying that first row by a^{-1}. Then:

1. *R and R' have the same row span;*
2. *the first row of R' is*

$$(1, c, a^{-1}g);$$

3. *after deleting the first hyperbolic coordinate pair from the successor rows of R', one obtains a tail family*
$$G' = \{g'_1, \dots, g'_m\} \subset K^n$$
such that
$$\langle a^{-1}g, a^{-1}g\rangle = 0, \qquad \langle a^{-1}g, g'_i\rangle = 0 \text{ for all } i, \qquad \langle g'_i, g'_j\rangle = 0 \text{ for all } i, j.$$

Proof. The equality of row spans is Proposition 3.19. The formula for the first row is the same proposition applied to the chosen pivot row.

Because row permutation and nonzero row scaling preserve pairwise orthogonality, the normalized family R' is still pairwise orthogonal. By construction its first row is $(1, c, a^{-1}g)$, and every successor row still has head on the same c-isotropic line. Therefore Theorem 3.23 applies to R'. Its conclusion is exactly that the deleted successor-tail family satisfies the displayed isotropy and orthogonality conditions. □

Corollary 3.25 (Existence of a pivot-normalized representative) *Under the hypotheses of Theorem 3.24, there exists a row family*
$$R' = \{R'_0, \dots, R'_m\} \subset K^{n+2}$$
such that:

1. *R' has the same row span as R;*
2. *$R'_0 = (1, c, x')$ for some $x' \in K^n$;*
3. *every successor row of R' has the form*
$$R'_i = (a'_i, ca'_i, g'_i);$$
4. *deleting the first hyperbolic coordinate pair from the successor rows yields a pivot-tail package relative to x'.*

Proof. Take R' to be the pivot-normalized family constructed in Theorem 3.24. The first three conclusions are part of that theorem's construction, and the last is exactly its tail-package conclusion. □

Corollary 3.26 (Existence of an explicit pivotRows representative) *Under the hypotheses of Theorem 3.24, there exist a pivot tail $x' \in K^n$, scalars $a'_1, \dots, a'_m \in K$, and a tail family*
$$G' = \{g'_1, \dots, g'_m\} \subset K^n$$
such that the row family
$$\widetilde{R} = \{(1, c, x'),\ (a'_1, ca'_1, g'_1),\ \dots,\ (a'_m, ca'_m, g'_m)\}$$
has the same row span as R, is pairwise orthogonal, and the tail data satisfy
$$\langle x', x'\rangle = 0, \qquad \langle x', g'_i\rangle = 0 \text{ for all } i, \qquad \langle g'_i, g'_j\rangle = 0 \text{ for all } i, j.$$

Proof. By Corollary 3.25, there is a same-row-span pivot-normalized representative R'. Writing its successor rows explicitly as

$$R'_i = (a'_i, ca'_i, g'_i)$$

produces a family $\widetilde{R}$ of the displayed form. The pairwise orthogonality and the tail-package identities then follow from Theorem 3.23 applied to R'. □

Proposition 3.27 (Pure-tail normal form for a pivotRows representative) *Let*

$$\widetilde{R} = \{(1, c, x),\ (a_1, ca_1, g_1), \ldots, (a_m, ca_m, g_m)\}$$

be a pivotRows family. Define residual tails

$$h_i = g_i - a_i x \qquad (1 \le i \le m),$$

and form the pure-tail family

$$\widetilde{R}^{\text{pure}} = \{(1, c, x),\ (0, 0, h_1), \ldots, (0, 0, h_m)\}.$$

Then $\widetilde{R}$ and $\widetilde{R}^{\text{pure}}$ have the same row span. If, in addition,

$$\langle x, x\rangle = 0, \qquad \langle x, g_i\rangle = 0, \qquad \langle g_i, g_j\rangle = 0,$$

then the residual tails satisfy

$$\langle x, h_i\rangle = 0 \qquad \textit{and} \qquad \langle h_i, h_j\rangle = 0$$

for all i, j, so $\widetilde{R}^{\text{pure}}$ is again pairwise orthogonal.

Proof. Each successor row decomposes as

$$(a_i, ca_i, g_i) = a_i(1, c, x) + (0, 0, g_i - a_i x).$$

Hence every row of $\widetilde{R}$ lies in the span of $\widetilde{R}^{\text{pure}}$, and conversely each pure-tail row is the difference of a successor row and a scalar multiple of the pivot row, so the row spans agree.

If the pivot-tail package identities hold, then

$$\langle x, h_i\rangle = \langle x, g_i\rangle - a_i\langle x, x\rangle = 0,$$

and similarly

$$\langle h_i, h_j\rangle = \langle g_i, g_j\rangle - a_j\langle g_i, x\rangle - a_i\langle x, g_j\rangle + a_i a_j\langle x, x\rangle = 0.$$

Therefore the pure tails remain pairwise orthogonal and orthogonal to the pivot tail. □

Corollary 3.28 (Existence of a same-code pure-tail representative) *Under the hypotheses of Theorem 3.24, there exist a pivot tail $x' \in K^n$, scalars $a'_1, \dots, a'_m \in K$, and tails $g'_1, \dots, g'_m \in K^n$ such that the family*

$$R^{\text{pure}} = \{(1, c, x'),\ (0, 0, g'_1), \dots, (0, 0, g'_m)\}$$

has the same row span as the original family R, is pairwise orthogonal, and its deleted tail family satisfies

$$\langle x', g'_i \rangle = 0 \qquad \text{and} \qquad \langle g'_i, g'_j \rangle = 0$$

for all i, j.

Proof. By Corollary 3.26, R admits a same-row-span pivotRows representative

$$\widetilde{R} = \{(1, c, x'),\ (a'_1, ca'_1, h_1), \dots, (a'_m, ca'_m, h_m)\}$$

whose deleted tail family is a pivot-tail package. Applying Proposition 3.27 to $\widetilde{R}$ replaces it, without changing the row span, by the pure-tail family

$$\{(1, c, x'),\ (0, 0, h_1 - a'_1 x'), \dots, (0, 0, h_m - a'_m x')\},$$

and the same proposition gives the required orthogonality statements for the deleted tail family. □

Proposition 3.29 (Obstruction to a naive exact same-code converse) *Let*

$$R^{\text{pure}} = \{(1, c, x),\ (0, 0, h_1), \dots, (0, 0, h_m)\}$$

with $c \neq 0$. Then every vector in the row span of R^{pure} has first two coordinates of the form $(\alpha, c\alpha)$. In particular, no vector of the form $(1, 0, *)$ lies in the same row span. Consequently, a pure-tail representative cannot already be an exact split `buildRows` *family in the original coordinates.*

Proof. Every generator row of R^{pure} has first two coordinates either $(1, c)$ or $(0, 0)$, so every linear combination has first two coordinates of the form $(\alpha, c\alpha)$. Since $c \neq 0$, such a vector can never equal $(1, 0, *)$. □

Theorem 3.30 (Head alignment to exact split building-up form) *Let R^{pure} be a pairwise orthogonal pure-tail representative*

$$R^{\text{pure}} = \{(1, c, x),\ (0, 0, h_1), \dots, (0, 0, h_m)\}.$$

After applying the explicit head transform

$$(v_0, v_1, \text{tail}) \longmapsto \left(\frac{v_0 - cv_1}{2}, \frac{v_0 + cv_1}{2}, \text{tail} \right),$$

the family becomes an exact split `buildRows` *family with the same deleted tail package. In particular, the obstruction in Proposition 3.29 is a coordinate obstruction, not an existential obstruction.*

Proof. The chosen head transform sends $(1, c)$ to $(1, 0)$ and fixes the zero head $(0, 0)$. Hence the distinguished row becomes $(1, 0, x)$, while every remaining row keeps the same tail h_i. Because the pure-tail family is pairwise orthogonal, each h_i satisfies $\langle x, h_i \rangle = 0$, so the aligned successor rows acquire exactly the zero head prescribed by the `buildRows` formula. Therefore the transformed family is precisely the normalized split `buildRows` family attached to x and the deleted tail package. □

Theorem 3.31 (Reverse theorem up to split-isometric equivalence) *Let R be a pairwise orthogonal family whose rows all lie on the c-isotropic line, and assume one row has nonzero head. Then, after passing to a same-code pivot-normalized representative and then to a pure-tail representative, there exist x, correction scalars a_i, and a tail family G such that R is split-isometrically equivalent to an exact split* `buildRows` *family*

$$\texttt{buildRows}(x, c, H), \qquad H = \texttt{pivotResidualTails}(x, a, G).$$

Equivalently, the reverse direction is formally valid up to the explicit split-isometry relation carried by the hyperbolic head plane.

Proof. The normalization front end produces a same-code pure-tail representative. Proposition 3.29 shows why one cannot stop there in the original coordinates, and Theorem 3.30 supplies the missing head alignment. The corresponding split isometry acts only on the distinguished head hyperbolic plane and preserves the ambient bilinear form, so self-orthogonality, self-duality, and the predecessor-successor relation are unchanged under this replacement. The resulting existence statement is exactly the split-isometric reverse theorem formalized in Lean; the ambient-linear-equivalence version is obtained as a corollary. □

Theorem 3.32 (Comparison with the classical Lee–Kim wording) *Fix $c \in K^\times$ with $c^2 = -1$, and let*

$$T_c(v_0, v_1, \mathrm{tail}) = \left(\frac{v_0 - cv_1}{2}, \frac{v_0 + cv_1}{2}, \mathrm{tail} \right).$$

Then T_c fixes the tail coordinates, sends the hyperbolic head pair

$$(1, c, \mathrm{tail}) \longmapsto (1, 0, \mathrm{tail}), \qquad (1, -c, \mathrm{tail}) \longmapsto (0, 1, \mathrm{tail}),$$

and is a split isometry from the Euclidean form on the distinguished 2-plane to the split hyperbolic form H_2. Consequently, the Lean theorem proving existence of a split `buildRows` *representative up to* `SplitIsometryCodeEquiv` *is precisely the reverse building-up theorem in Lee–Kim style after a fixed hyperbolic basis change on the first coordinate pair.*

Proof. The first statement is exactly the explicit formula defining the head transform: the isotropic vector $(1, c)$ is sent to the standard pivot row head $(1, 0)$, while the opposite hyperbolic partner $(1, -c)$ is sent to $(0, 1)$. The same linear map is also proved to be a split isometry for the hyperbolic head form. Therefore the formal reverse theorem says precisely that, after the fixed hyperbolic basis change T_c on the distinguished 2-plane, the reverse direction lands in the standard `buildRows` normal form. This is the Lee–Kim converse rewritten in the hyperbolic basis language isolated in the present paper. □

Theorem 3.33 (Rowwise characterization of normalized split build families) *Fix $x \in K^n$ and $c \in K$, and let*

$$R = \{R_0, R_1, \dots, R_m\} \subset K^{n+2}$$

be a row family with $R_0 = (1, 0, x)$. Let

$$\tau : K^{n+2} \to K^n, \qquad \tau(a, b, g) = g$$

denote deletion of the first two coordinates. Then the following are equivalent:

1. *For every $i \geq 1$, the row R_i has a c-isotropic head and is orthogonal to R_0, that is,*

$$R_i = (a_i, ca_i, g_i) \quad \text{for some } a_i \in K,\ g_i \in K^n, \qquad \langle R_0, R_i \rangle = 0.$$

2. *For every $i \geq 1$, the row R_i is exactly the canonical building-up row determined by its tail:*

$$R_i = \big(-\langle x, \tau(R_i)\rangle,\ -c\langle x, \tau(R_i)\rangle,\ \tau(R_i)\big).$$

Proof. Suppose first that $R_i = (a_i, ca_i, g_i)$ and $\langle R_0, R_i \rangle = 0$. Since $R_0 = (1, 0, x)$, expanding the dot product gives

$$0 = \langle R_0, R_i \rangle = a_i + \langle x, g_i \rangle.$$

Hence $a_i = -\langle x, g_i \rangle$, so

$$R_i = (-\langle x, g_i \rangle,\ -c\langle x, g_i \rangle,\ g_i).$$

Since $g_i = \tau(R_i)$, this is precisely the stated canonical form.

Conversely, if

$$R_i = \big(-\langle x, \tau(R_i)\rangle,\ -c\langle x, \tau(R_i)\rangle,\ \tau(R_i)\big),$$

then R_i has a c-isotropic head by construction, and

$$\langle R_0, R_i \rangle = -\langle x, \tau(R_i)\rangle + \langle x, \tau(R_i)\rangle = 0.$$

Thus the two conditions are equivalent. □

Proposition 3.34 (Unique parent and unique child in normalized split form) *Fix $x \in K^n$, $c \in K$, and a parent row family*

$$G = \{G_1, \dots, G_m\} \subset K^n.$$

Then there exists a unique normalized split build family

$$R = \{R_0, R_1, \dots, R_m\} \subset K^{n+2}$$

such that

$$R_0 = (1, 0, x) \qquad \text{and} \qquad \tau(R_i) = G_i \quad (1 \leq i \leq m).$$

It is given explicitly by

$$R_0 = (1, 0, x), \qquad R_i = \big(-\langle x, G_i \rangle,\ -c\langle x, G_i \rangle,\ G_i\big) \quad (1 \leq i \leq m).$$

Conversely, every normalized split build family R has a unique parent family, namely the tail family $\tau(R_i)$ obtained by deleting the first row and the first hyperbolic coordinate pair.

Proof. For existence, define R by the displayed formula. This is the building-up prescription, so R is a normalized split build family with parent G. For uniqueness, let R' be any normalized split build family with the same parent G. By Theorem 3.33, each successor row of R' is uniquely determined by its tail:

$$R'_i = \big(-\langle x, \tau(R'_i)\rangle,\ -c\langle x, \tau(R'_i)\rangle,\ \tau(R'_i)\big) = \big(-\langle x, G_i\rangle,\ -c\langle x, G_i\rangle,\ G_i\big).$$

Hence $R' = R$. For the converse, deleting the first row and the first hyperbolic coordinate pair produces a row family G, and this G is uniquely determined by the rows of R. □

Remark 3.35 The Lean file `BuildingUpFormalization.lean` certifies four layers of the split theory: the split arithmetic of the head plane and the norm form, the forward building-up theorem through self-duality, the normalized reverse chain through pure-tail reduction and the naive-converse obstruction, and the explicit head-alignment step yielding the reverse theorem under `SplitIsometryCodeEquiv`. Thus the formalization certifies the reverse existence statement in the precise split-isometric sense used in this paper.

Theorem 3.36 (One-step cohomological comparison between Lagrangian reduction and binary building-up) *Let*

$$W = \bigoplus_{v\in S} H^1_{\text{ét}}(K_v, \mu_2)$$

be the Chinburg–Zhang ambient space, equipped with the bilinear form induced by the local cup products followed by the localization boundary map

$$\delta_2 : \bigoplus_{v\in S} H^2_{\text{ét}}(K_v, \mu_2) \longrightarrow H^3_c(U, \mu_2) \cong \mathbb{F}_2.$$

Assume we are in the binary Euclidean case of Corollary 1.4, so that after choosing a Euclidean basis we may identify $W \cong \mathbb{F}_2^{2n}$. Let

$$L := \operatorname{im}\Phi \subset W$$

be the associated arithmetic self-dual code.

Suppose L is represented, up to equivalence, by a boxed generator matrix $\widetilde{M}$, and that $\widetilde{M}$ is written in the block form

$$\widetilde{M} = \begin{pmatrix} 01 & u \\ w & M' \end{pmatrix},$$

where w consists only of identical pairs and M' generates a self-dual code $L' \subset \mathbb{F}_2^{2n-2}$.

Then the following hold.

1. *There is an orthogonal decomposition*

$$W \cong H \perp W',$$

where $H \cong \mathbb{F}_2^2$ is the hyperbolic plane corresponding to the first block-pair and $W' \cong \mathbb{F}_2^{2n-2}$ is the complementary Euclidean space.

2. *Under this decomposition, $L' \subset W'$ is a Lagrangian subspace.*

3. *Writing the first row of $\widetilde{M}$ as*

$$r_0 = (1, 0, x), \qquad x \in W',$$

and the remaining rows as

$$r_i = (y_i, y_i, g_i), \qquad g_i \in L',\ y_i \in \mathbb{F}_2,$$

one has

$$y_i = \langle x, g_i \rangle \qquad \text{for all } i.$$

4. *Consequently,*

$$L = \mathrm{Span}\{\, r_0,\ r_i\ (1 \le i \le n-1)\,\}$$

is obtained from L' by one Kim building-up step. Equivalently, the Chinburg–Zhang boxed reduction

$$L \rightsquigarrow L'$$

is the inverse, up to equivalence, of the Kim building-up extension

$$L' \rightsquigarrow L.$$

In particular, after boxed normalization, the Chinburg–Zhang reduction and the Kim extension identify the same one-step predecessor-successor relation on binary Lagrangian codes after the associated equivalence operations.

Proof. By Chinburg–Zhang Theorem 1.1 [12], the image

$$L = \mathrm{im}\,\Phi \subset W$$

is its own orthogonal complement with respect to the bilinear form coming from the local cup products and the boundary map δ_2. Hence L is a maximal isotropic subspace of W, i.e. a Lagrangian.

Now choose a Euclidean basis as in Corollary 1.4 and identify $W \cong \mathbb{F}_2^{2n}$. By the proof of Chinburg–Zhang Theorem 1.5, after row operations and coordinate permutations one may assume that the boxed matrix has the block form

$$\widetilde{M} = \begin{pmatrix} 01 & u \\ w & M' \end{pmatrix},$$

where w consists only of identical pairs and M' generates a self-dual code of length $2n-2$. Let $H \subset W$ be the span of the first block-pair and let W' be the span of the remaining coordinates. Since the ambient basis is Euclidean, the first block-pair has Gram matrix

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$

so H is a hyperbolic plane and

$$W \cong H \perp W'.$$

Because M' generates a self-dual code of length $2n-2$, its row span $L' \subset W'$ is again maximal isotropic, hence Lagrangian. This proves (1) and (2).

Write the first row of $\widetilde{M}$ as

$$r_0 = (1, 0, x), \qquad x \in W',$$

and write every other row in the form

$$r_i = (y_i, y_i, g_i),$$

which is possible because w consists only of identical pairs. Since L is self-orthogonal, for each i we have

$$0 = \langle r_0, r_i \rangle.$$

Expanding this inner product gives

$$0 = \langle (1, 0, x), (y_i, y_i, g_i) \rangle = y_i + \langle x, g_i \rangle.$$

Over $\mathbb{F}_2$, this implies

$$y_i = \langle x, g_i \rangle.$$

Thus every row r_i has the form

$$r_i = (\langle x, g_i \rangle,\ \langle x, g_i \rangle,\ g_i).$$

But this is exactly the binary Kim building-up pattern: starting from the shorter self-dual code $L' \subset W'$ with generator rows g_i, and adjoining the extra row

$$r_0 = (1, 0, x),$$

the remaining rows are corrected by the scalars

$$y_i = \langle x, g_i \rangle$$

to produce a self-dual code in $H \perp W' \cong \mathbb{F}_2^{2n}$. Therefore L is obtained from L' by one building-up extension step.

Hence the Chinburg–Zhang boxed reduction

$$L \rightsquigarrow L'$$

and the Kim building-up construction

$$L' \rightsquigarrow L$$

match at the one-step level after boxed normalization and the same equivalence operations used in both theories. This is the precise comparison asserted in the theorem. $\square$

Corollary 3.37 *After boxed normalization, the Chinburg–Zhang boxed-matrix induction and the Kim building-up construction may be identified at the one-step level as top-down and bottom-up descriptions of the same predecessor-successor relation.*

Listing 10: Lean verification for Theorem 3.36.

```
import Mathlib

open Matrix
open BigOperators

section Base

variable {K : Type*} [Field K]

/-!
Core binary building-up / boxed-reduction infrastructure.
The characteristic-2 assumptions are only used in the orthogonality part.
-/

def dot {n : ℕ} (u v : Fin n → K) : K :=
  ∑ j, u j * v j

theorem dot_comm {n : ℕ} (u v : Fin n → K) :
    dot u v = dot v u := by
  unfold dot
  apply Finset.sum_congr rfl
  intro j hj
  ring

def head2 (a b : K) : Fin 2 → K
  | <0, _> => a
  | <1, _> => b

def prepend2 {n : ℕ} (a b : K) (u : Fin n → K) : Fin (2 + n) → K :=
  Fin.append (head2 a b) u

def r0 {n : ℕ} (x : Fin n → K) : Fin (2 + n) → K :=
  prepend2 1 0 x

def riBin {n : ℕ} (yi : K) (g : Fin n → K) : Fin (2 + n) → K :=
  prepend2 yi yi g

/-- A boxed-reduction family with explicit coefficients `Y i`. -/
def boxedFamily {m n : ℕ}
    (x : Fin n → K) (Y : Fin m → K) (G : Fin m → Fin n → K) :
    Fin (m + 1) → Fin (2 + n) → K :=
```

```
  Fin.cases (r0 x) (fun i => riBin (Y i) (G i))

/-- The Kim building-up family, where the coefficient is `dot x (G i)`. -/
def buildRowsBin {m n : ℕ}
    (x : Fin n → K) (G : Fin m → Fin n → K) :
    Fin (m + 1) → Fin (2 + n) → K :=
  Fin.cases (r0 x) (fun i => riBin (dot x (G i)) (G i))

theorem boxedFamily_eq_buildRowsBin
    {m n : ℕ}
    {x : Fin n → K} {Y : Fin m → K} {G : Fin m → Fin n → K}
    (hY : ∀ i : Fin m, Y i = dot x (G i)) :
    boxedFamily x Y G = buildRowsBin x G := by
  funext i
  rcases Fin.eq_zero_or_eq_succ i with rfl | <j, rfl>
  · simp [boxedFamily, buildRowsBin]
  · simp [boxedFamily, buildRowsBin, hY j]

/-- Tail of a vector in `K^(2+n)` obtained by deleting the first two coordinates. -/
def tailVec {n : ℕ} (v : Fin (2 + n) → K) : Fin n → K :=
  fun j => v (Fin.natAdd 2 j)

/-- Delete the first row and the first hyperbolic coordinate pair. -/
def deleteHyperbolicPair {m n : ℕ}
    (R : Fin (m + 1) → Fin (2 + n) → K) : Fin m → Fin n → K :=
  fun i => tailVec (R (Fin.succ i))

theorem tailVec_prepend2
    {n : ℕ} (a b : K) (u : Fin n → K) :
    tailVec (prepend2 a b u) = u := by
  funext j
  simp [tailVec, prepend2]

theorem tailVec_r0
    {n : ℕ} (x : Fin n → K) :
    tailVec (r0 x) = x := by
  simpa [r0] using tailVec_prepend2 (K := K) 1 0 x

theorem tailVec_riBin
    {n : ℕ} (yi : K) (g : Fin n → K) :
    tailVec (riBin yi g) = g := by
  simpa [riBin] using tailVec_prepend2 (K := K) yi yi g

theorem buildRowsBin_zero
    {m n : ℕ} (x : Fin n → K) (G : Fin m → Fin n → K) :
    buildRowsBin x G 0 = r0 x := by
  simp [buildRowsBin]
```

```
theorem buildRowsBin_succ
    {m n : ℕ} (x : Fin n → K) (G : Fin m → Fin n → K) (i : Fin m) :
    buildRowsBin x G (Fin.succ i) = riBin (dot x (G i)) (G i) := by
  simp [buildRowsBin]

/-- Deleting the distinguished hyperbolic pair from a Kim building-up family
recovers the original shorter family. -/
theorem deleteHyperbolicPair_buildRowsBin
    {m n : ℕ} (x : Fin n → K) (G : Fin m → Fin n → K) :
    deleteHyperbolicPair (buildRowsBin x G) = G := by
  funext i
  funext j
  rw [deleteHyperbolicPair]
  simp [tailVec, buildRowsBin, riBin, prepend2]

/-- If a family has the Kim/boxed pattern row-by-row, then rebuilding from its
deleted hyperbolic pair recovers the original family. -/
theorem buildRowsBin_deleteHyperbolicPair_eq
    {m n : ℕ} {x : Fin n → K} {R : Fin (m + 1) → Fin (2 + n) → K}
    (h0 : R 0 = r0 x)
    (hs : ∀ i : Fin m,
      R (Fin.succ i) = riBin (dot x (tailVec (R (Fin.succ i)))) (tailVec (R (Fin.succ
    i)))) :
    buildRowsBin x (deleteHyperbolicPair R) = R := by
  funext i
  rcases Fin.eq_zero_or_eq_succ i with rfl | <j, rfl>
  .
    rw [buildRowsBin_zero]
    exact h0.symm
  .
    rw [buildRowsBin_succ]
    change
      riBin (dot x (tailVec (R (Fin.succ j)))) (tailVec (R (Fin.succ j))) =
        R (Fin.succ j)
    exact (hs j).symm

/-- A row family is in boxed/Kim normal form if its zero-th row is `r0 x` and each
successor row is of the form `(dot x g, dot x g, g)`. -/
def IsBoxedKimFamily {m n : ℕ}
    (x : Fin n → K) (R : Fin (m + 1) → Fin (2 + n) → K) : Prop :=
  R 0 = r0 x ∧
  ∀ i : Fin m,
    R (Fin.succ i) =
      riBin (dot x (tailVec (R (Fin.succ i)))) (tailVec (R (Fin.succ i)))

theorem rebuild_of_IsBoxedKimFamily
```

```
    {m n : ℕ} {x : Fin n → K} {R : Fin (m + 1) → Fin (2 + n) → K}
    (hR : IsBoxedKimFamily x R) :
    buildRowsBin x (deleteHyperbolicPair R) = R := by
  rcases hR with <h0, hs>
  exact buildRowsBin_deleteHyperbolicPair_eq h0 hs

/-- The linear code generated by the rows of `R`. -/
def rowSpace {m n : ℕ} (R : Fin m → Fin n → K) : Submodule K (Fin n → K) :=
  Submodule.span K (Set.range R)

/-- Permute coordinates of a vector. -/
def permuteVec {n : ℕ} σ( : Equiv.Perm (Fin n)) (v : Fin n → K) : Fin n → K :=
  fun j => v σ( j)

/-- Permute coordinates of every row in a row family. -/
def permuteFamily {m n : ℕ} σ( : Equiv.Perm (Fin n)) (R : Fin m → Fin n → K) :
    Fin m → Fin n → K :=
  fun i => permuteVec σ (R i)

/-- Two row families generate the same code. -/
def SameCode {₁m ₂m n : ℕ}
    (R : Fin ₁m → Fin n → K) (S : Fin ₂m → Fin n → K) : Prop :=
  rowSpace R = rowSpace S

/-- Code equivalence: equality of row spaces up to a coordinate permutation. -/
def CodeEquiv {₁m ₂m n : ℕ}
    (R : Fin ₁m → Fin n → K) (S : Fin ₂m → Fin n → K) : Prop :=
  ∃ σ : Equiv.Perm (Fin n), SameCode (permuteFamily σ R) S

theorem permuteVec_refl {n : ℕ} (v : Fin n → K) :
    permuteVec (Equiv.refl (Fin n)) v = v := by
  funext j
  rfl

theorem permuteFamily_refl {m n : ℕ} (R : Fin m → Fin n → K) :
    permuteFamily (Equiv.refl (Fin n)) R = R := by
  funext i
  exact permuteVec_refl (R i)

theorem sameCode_refl {m n : ℕ} (R : Fin m → Fin n → K) : SameCode R R := rfl

theorem sameCode_of_eq
    {m n : ℕ} {R S : Fin m → Fin n → K} (h : R = S) :
    SameCode R S := by
  cases h
  rfl
```

```
theorem codeEquiv_of_sameCode
    {₁m ₂m n : ℕ}
    {R : Fin ₁m → Fin n → K} {S : Fin ₂m → Fin n → K}
    (h : SameCode R S) : CodeEquiv R S := by
  refine <Equiv.refl (Fin n), ?_>
  simpa [CodeEquiv, SameCode, permuteFamily, permuteVec] using h

theorem codeEquiv_refl {m n : ℕ} (R : Fin m → Fin n → K) : CodeEquiv R R := by
  exact codeEquiv_of_sameCode (sameCode_refl R)

theorem sameCode_delete_buildRowsBin
    {m n : ℕ} (x : Fin n → K) (G : Fin m → Fin n → K) :
    SameCode (deleteHyperbolicPair (buildRowsBin x G)) G := by
  apply sameCode_of_eq
  exact deleteHyperbolicPair_buildRowsBin x G

theorem codeEquiv_delete_buildRowsBin
    {m n : ℕ} (x : Fin n → K) (G : Fin m → Fin n → K) :
    CodeEquiv (deleteHyperbolicPair (buildRowsBin x G)) G := by
  exact codeEquiv_of_sameCode (sameCode_delete_buildRowsBin x G)

theorem sameCode_rebuild_of_IsBoxedKimFamily
    {m n : ℕ} {x : Fin n → K} {R : Fin (m + 1) → Fin (2 + n) → K}
    (hR : IsBoxedKimFamily x R) :
    SameCode (buildRowsBin x (deleteHyperbolicPair R)) R := by
  apply sameCode_of_eq
  exact rebuild_of_IsBoxedKimFamily hR

theorem codeEquiv_rebuild_of_IsBoxedKimFamily
    {m n : ℕ} {x : Fin n → K} {R : Fin (m + 1) → Fin (2 + n) → K}
    (hR : IsBoxedKimFamily x R) :
    CodeEquiv (buildRowsBin x (deleteHyperbolicPair R)) R := by
  exact codeEquiv_of_sameCode (sameCode_rebuild_of_IsBoxedKimFamily hR)

theorem sameCode_boxedFamily_buildRowsBin
    {m n : ℕ}
    {x : Fin n → K} {Y : Fin m → K} {G : Fin m → Fin n → K}
    (hY : ∀ i : Fin m, Y i = dot x (G i)) :
    SameCode (boxedFamily x Y G) (buildRowsBin x G) := by
  apply sameCode_of_eq
  exact boxedFamily_eq_buildRowsBin hY

theorem codeEquiv_boxedFamily_buildRowsBin
    {m n : ℕ}
    {x : Fin n → K} {Y : Fin m → K} {G : Fin m → Fin n → K}
    (hY : ∀ i : Fin m, Y i = dot x (G i)) :
    CodeEquiv (boxedFamily x Y G) (buildRowsBin x G) := by
```

```
  exact codeEquiv_of_sameCode (sameCode_boxedFamily_buildRowsBin hY)

end Base

section CharTwoPart

variable {K : Type*} [Field K] [CharP K 2]

theorem dot_prepend2_prepend2
    {n : ℕ} (a b a' b' : K) (u v : Fin n → K) :
    dot (prepend2 a b u) (prepend2 a' b' v)
      = a * a' + b * b' + dot u v := by
  unfold dot prepend2
  rw [Fin.sum_univ_add]
  simp [head2, Fin.sum_univ_two, add_assoc]

theorem two_eq_zero : (2 : K) = 0 := by
  exact CharP.cast_eq_zero (R := K) 2

theorem dot_r0_r0_bin
    {n : ℕ} {x : Fin n → K}
    (hx : dot x x = (1 : K)) :
    dot (r0 x) (r0 x) = 0 := by
  rw [r0, dot_prepend2_prepend2, hx]
  calc
    (1 : K) * 1 + 0 * 0 + 1 = (2 : K) := by ring
    _ = 0 := two_eq_zero

theorem dot_r0_riBin
    {n : ℕ} {x g : Fin n → K} {yi : K}
    (hyi : yi = dot x g) :
    dot (r0 x) (riBin yi g) = 0 := by
  rw [r0, riBin, dot_prepend2_prepend2, hyi]
  calc
    (1 : K) * dot x g + 0 * dot x g + dot x g = (2 : K) * dot x g := by ring
    _ = 0 := by rw [two_eq_zero, zero_mul]

theorem dot_riBin_riBin
    {n : ℕ} {g h : Fin n → K} {yi yj : K}
    (hgh : dot g h = 0) :
    dot (riBin yi g) (riBin yj h) = 0 := by
  rw [riBin, riBin, dot_prepend2_prepend2, hgh]
  calc
    yi * yj + yi * yj + 0 = (2 : K) * (yi * yj) := by ring
    _ = 0 := by rw [two_eq_zero, zero_mul]

theorem buildRowsBin_zero_zero
```

```
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (hx : dot x x = (1 : K)) :
    dot (buildRowsBin x G 0) (buildRowsBin x G 0) = 0 := by
  simp [buildRowsBin]
  exact dot_r0_r0_bin hx

theorem buildRowsBin_zero_succ
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (i : Fin m) :
    dot (buildRowsBin x G 0) (buildRowsBin x G (Fin.succ i)) = 0 := by
  simp [buildRowsBin]
  exact dot_r0_riBin rfl

theorem buildRowsBin_succ_zero
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (i : Fin m) :
    dot (buildRowsBin x G (Fin.succ i)) (buildRowsBin x G 0) = 0 := by
  rw [dot_comm]
  exact buildRowsBin_zero_succ i

theorem buildRowsBin_succ_succ
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0)
    (i j : Fin m) :
    dot (buildRowsBin x G (Fin.succ i)) (buildRowsBin x G (Fin.succ j)) = 0 := by
  simp [buildRowsBin]
  exact dot_riBin_riBin (hgh := hG i j)

theorem buildRowsBin_orthogonal
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (hx : dot x x = (1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    ∀ i j : Fin (m + 1), dot (buildRowsBin x G i) (buildRowsBin x G j) = 0 := by
  intro i j
  rcases Fin.eq_zero_or_eq_succ i with rfl | <i, rfl>
  · rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
    · exact buildRowsBin_zero_zero hx
    · exact buildRowsBin_zero_succ j
  · rcases Fin.eq_zero_or_eq_succ j with rfl | <j, rfl>
    · exact buildRowsBin_succ_zero i
    · exact buildRowsBin_succ_succ (hG := hG) i j

def rowMatrix {m n : ℕ} (R : Fin m → Fin n → K) : Matrix (Fin m) (Fin n) K :=
  fun i j => R i j

theorem rowMatrix_mul_transpose_apply
    {m n : ℕ} (R : Fin m → Fin n → K) (i j : Fin m) :
```

```
    (rowMatrix R ⬝ (rowMatrix R).transpose) i j = dot (R i) (R j) := by
  unfold rowMatrix dot
  rw [Matrix.mul_apply]
  simp

theorem buildRowsBin_matrix_gram_zero
    {m n : ℕ} {x : Fin n → K} {G : Fin m → Fin n → K}
    (hx : dot x x = (1 : K))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    rowMatrix (buildRowsBin x G) ⬝ (rowMatrix (buildRowsBin x G)).transpose = 0 := by
  ext i j
  rw [rowMatrix_mul_transpose_apply]
  exact buildRowsBin_orthogonal (hx := hx) (hG := hG) i j

theorem boxedFamily_matrix_gram_zero
    {m n : ℕ}
    {x : Fin n → K} {Y : Fin m → K} {G : Fin m → Fin n → K}
    (hx : dot x x = (1 : K))
    (hY : ∀ i : Fin m, Y i = dot x (G i))
    (hG : ∀ i j : Fin m, dot (G i) (G j) = 0) :
    rowMatrix (boxedFamily x Y G) ⬝ (rowMatrix (boxedFamily x Y G)).transpose = 0 :=
    by
  rw [boxedFamily_eq_buildRowsBin hY]
  exact buildRowsBin_matrix_gram_zero (hx := hx) (hG := hG)

end CharTwoPart
```

4 Constructive Classification over GF(5)

The computational results over GF(5) should be read constructively, not merely statistically. The building-up procedure produces explicit child codes from a parent code and an extension vector, and the small-length classification records how this mechanism behaves in practice.

4.1 Complete classification up to length 8

Theorem 4.1 (Complete classification over GF(5) up to length 8) *Up to monomial equivalence, the self-dual codes over* GF(5) *of lengths* 2, 4, 6, *and* 8 *are classified as follows:*

Length	*Parameters*	*# Weight-Enumerator Types*	*# Inequiv. Classes*	*Minimum Distances*
2	$[2,1]$	*1*	*1*	$d=2$
4	$[4,2]$	*1*	*1*	$d=2$
6	$[6,3]$	*2*	*2*	$d=2,4$
8	$[8,4]$	*3*	*53*	$d=2,4$

For lengths 2 *and* 4*, each weight-enumerator type consists of a single equivalence class. For length* 6*, the two weight-enumerator types each form a single equivalence class. For length* 8*, the* 53 *inequivalent classes split into three weight-enumerator types of sizes* 5*,* 16*, and* 32*.*

Proof. The length-2 and length-4 cases are unique up to monomial equivalence. For length 6, the reverse theory developed in Section 3 reduces the search to the unique self-dual $[4,2]$ parent together with all admissible extension vectors $x \in \mathrm{GF}(5)^4$ satisfying $\langle x,x\rangle = -1$. There are exactly 120 such vectors. Exhaustive building-up on those 120 inputs, followed by grouping by minimum distance and weight enumerator, leaves a finite candidate list. Exact monomial equivalence is then tested inside each bucket by explicit orbit reduction under the finite monomial group generated by coordinate permutations and nonzero coordinate scalings, so the final step is exact rather than heuristic. This leaves exactly two classes.

For length 8, one starts from the two classified self-dual $[6,3]$ parents and again exhausts all admissible extension vectors. In this case each parent admits exactly 3100 such vectors, so the search processes 6200 generated children before reduction. Grouping first by minimum distance and weight enumerator again leaves finitely many buckets, and exact monomial equivalence is tested inside each bucket by the same explicit orbit reduction under coordinate permutations and nonzero coordinate scalings. This yields exactly 53 inequivalent classes, split into the three announced weight-enumerator types of sizes 5, 16, and 32. Since every admissible parent-extension pair is inspected and the final reduction step is exact, the listed counts are complete. □

4.2 Constructive interpretation

The formal and computational picture can be summarized very briefly. Choose $c \in \mathbb{F}_q$ with $c^2 = -1$, start from a self-dual parent code $C_0 = \mathrm{Span}(G_0)$ of length $2n$, pick an extension vector x with $\langle x,x\rangle = -1$, compute $y_i = \langle x, g_i\rangle$ for the rows g_i of G_0, and output the child family with distinguished row $(1,0,x)$ and successor rows $(-y_i, -cy_i, g_i)$. Over $\mathrm{GF}(5)$ this recipe produces all observed codes in the small-length classification.

At length 6, the unique $[4,2]$ parent yields exactly two inequivalent self-dual $[6,3]$ codes from the 120 valid extension vectors: a $d=2$ type obtained from 40 vectors and a $d=4$ type obtained from 80 vectors. At length 8, the two inequivalent self-dual $[6,3]$ parents yield the 53 inequivalent self-dual $[8,4]$ codes, partitioned into three weight-enumerator types of sizes 5, 16, and 32. Thus each observed type is constructively accessible through explicit

parent and extension-vector data, even when we suppress the full representative lists in the submission version.

Proposition 4.2 (Compact representative data for the GF(5) types) *For the two self-dual* $[6,3]$ *codes over* GF(5)*, one may take the representatives*

$$G_A = \begin{pmatrix} 1 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 2 \\ 0 & 0 & 1 & 2 & 1 & 2 \end{pmatrix}, \qquad G_B = \begin{pmatrix} 1 & 0 & 0 & 1 & 2 & 3 \\ 1 & 2 & 1 & 0 & 2 & 0 \\ 3 & 1 & 0 & 1 & 0 & 2 \end{pmatrix}.$$

Their weight enumerators are

$$W_A(z) = 1 + 12z^2 + 48z^4 + 64z^6, \qquad W_B(z) = 1 + 60z^4 + 24z^5 + 40z^6.$$

For the three self-dual $[8,4]$ *weight-enumerator types, one may take standard-form representatives* $G = [I_4 \mid A]$ *with*

$$A_I = \begin{pmatrix} 0&0&0&2 \\ 0&0&2&0 \\ 0&2&0&0 \\ 2&0&0&0 \end{pmatrix}, \qquad A_{II} = \begin{pmatrix} 0&0&0&2 \\ 1&2&2&0 \\ 2&1&3&0 \\ 2&3&1&0 \end{pmatrix}, \qquad A_{III} = \begin{pmatrix} 0&1&2&2 \\ 1&0&2&3 \\ 2&2&4&0 \\ 2&3&0&1 \end{pmatrix}.$$

Their weight enumerators are

$$W_I(z) = 1 + 16z^2 + 96z^4 + 256z^6 + 256z^8,$$
$$W_{II}(z) = 1 + 4z^2 + 60z^4 + 24z^5 + 280z^6 + 96z^7 + 160z^8,$$
$$W_{III}(z) = 1 + 48z^4 + 32z^5 + 288z^6 + 128z^7 + 128z^8.$$

Here Type I is decomposable, Type II is indecomposable with $d = 2$*, and Type III is the optimal* $d = 4$ *family.*

Proof. For the length-6 matrices, a direct Gram-matrix computation shows that both G_A and G_B generate self-dual codes. Moreover, G_A is decomposable, while G_B has minimum distance 4; equivalently, their displayed weight enumerators are different, so the two codes are inequivalent and represent the two length-6 classes.

For length 8, each displayed standard-form matrix $G = [I_4 \mid A]$ satisfies $AA^T = -I_4$, hence generates a self-dual code. The three displayed weight enumerators are pairwise different, so the corresponding codes are pairwise inequivalent. Since Theorem 4.1 shows that there are exactly three weight-enumerator types at length 8, the matrices A_I, A_{II}, A_{III} represent those three types. □

Remark 4.3 (Computational verification protocol) *The classification counts are produced by the following finite procedure. For each parent code we enumerate all extension vectors* x *with* $\langle x, x\rangle = -1$*, generate the child codes by the building-up construction, group them first by cheap invariants such as minimum distance and weight enumerator, and then perform exact monomial-equivalence reduction inside each bucket by explicit orbit reduction under coordinate permutations and nonzero coordinate scalings. Thus the numbers* 2*,* 53*,* 40/80*, and* 300/1200/1600*,* 0/600/2500 *come from exhaustive generation plus exact equivalence testing rather than from heuristic sampling. In the present submission we do not claim a separate mass-formula or automorphism-orbit proof, only this exhaustive protocol together with the representative data displayed above.*

4.3 Branching law

Proposition 4.4 (Branching law for building-up over GF(5)) *Let C_1 and C_2 denote the two inequivalent self-dual $[6,3]$ codes over GF(5), with minimum distances 2 and 4, respectively. Among the 3100 valid extension vectors for each parent, the child types are distributed as follows:*

Parent code	*Type I*	*Type II*	*Type III*
C_1 *(*$[6,3,2]$*)*	*300*	*1200*	*1600*
C_2 *(*$[6,3,4]$*)*	*0*	*600*	*2500*

Thus Type I occurs only from the $d = 2$ parent, whereas Type III occurs from both parents and dominates the output. In particular, minimum distance is not monotone under building-up: a $d = 4$ parent can produce both $d = 2$ and $d = 4$ children.

Proof. The table is obtained by exhaustive enumeration of all valid extension vectors for each parent and sorting the resulting $[8,4]$ children by weight-enumerator type. The vanishing of the Type I column for the $d = 4$ parent shows parent sensitivity, while the large Type III counts for both parents show that optimal children are robustly accessible. □

Remark 4.5 This branching law is the key constructive feature of the GF(5) computation: the building-up construction is not merely an existence tool, but a nontrivial transition mechanism on code types. The disappearance of Type I descendants from the $d = 4$ parent is consistent with the absence of the low-weight decomposable configuration needed to create a decomposable child under split extension. By contrast, the $d = 2$ parent admits such low-weight patterns, which explains the observed asymmetry in the branching table while leaving the optimal Type III family accessible from both parents. We record this as a structural heuristic rather than as an independent theorem.

Other split fields. The same norm-witness and building-up pipeline also produces explicit self-dual examples over larger split fields such as GF(13), GF(17), and GF(29). We omit the corresponding matrices here, since the present submission uses these computations only as supporting evidence that the split-field mechanism persists uniformly beyond GF(5).

5 Lean Formalization Summary

The current formalization is integrated in the single file `BuildingUpFormalization.lean`. It contains 247 kernel-certified theorems with zero `sorry`. On the forward side it formalizes the split arithmetic, the build-up rows, the linear-independence package for basis-indexed parent data, and the resulting self-duality theorem. On the reverse side it formalizes normalization,

pure-tail reduction, the obstruction to a naive exact same-code converse, the explicit head-alignment step, and the reverse existence theorem up to split-isometric equivalence.

The GF(5) tables in Section 4 remain computational outputs, but the constructive framework used to interpret them is now theorem-level Lean material.

6 Conclusion

We have studied self-dual codes over split finite fields $\mathbb{F}_q$ with $q \equiv 1 \pmod 4$ from three complementary viewpoints: the classical building-up construction, the hyperbolic geometry forced by the split condition $-1 \in (K^\times)^2$, and a theorem-level Lean 4 formalization of the resulting algebraic infrastructure. On the computational side, this yields the complete GF(5) classification for lengths $2, 4, 6, 8$ together with the branching law from length six to length eight. On the formal side, the integrated development now contains 247 kernel-certified theorems with zero `sorry`.

The single arithmetic fact $-1 \in (K^\times)^2$ governs the whole picture: it makes the distinguished Euclidean 2-plane hyperbolic, splits the norm form $x^2 + y^2$, explains the correction rows as motion along isotropic lines, removes the basic existence obstruction for the extension vectors, and supplies the head-alignment map used in the reverse theorem. In that sense the building-up construction is not an isolated recipe but the visible coding-theoretic manifestation of the split hyperbolic geometry.

For the reverse direction, the formalization establishes the normalization front end, the pure-tail reduction, the obstruction to a naive exact same-code converse, the explicit head-alignment step, and the resulting exact `buildRows` representative after the correct hyperbolic basis change. The converse is therefore established in a precise split-isometric sense, and the comparison theorem explains how this matches the classical Lee–Kim wording.

More broadly, the present work shows that the building-up construction is not merely an existence method. It is a constructive mechanism whose algebraic constraints, classification output, and normalization behavior can be expressed at theorem level and checked by the Lean kernel. That combination of explicit code-theoretic computation with machine-verified structural algebra is the main outcome of the paper.

Acknowledgments

The Lean 4 formalization was developed using Mathlib (leanprover/lean4:v4.28.0-rc1). The exhaustive enumeration of self-dual codes was performed by computational scripts. The integrated file `BuildingUpFormalization.lean` contains 247 kernel-certified theorems with zero `sorry`.

References

[1] C. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[2] E. Rains and N. J. A Sloane, Self-dual codes, in: V. S. Pless, W. C. Huffman (Eds.), Handbook of Coding Theory, Elsevier, Amsterdam. The Netherlands. (1998)

[3] C. Bachoc and P. Gaborit, "Designs and self-dual codes with long shadows," *J. Combin. Theory Ser. A,* 2004, 105(1): 15-34.

[4] E. Bannai, S. T. Dougherty, M. Harada, and M. Oura, Type II codes, "even unimodular lattices, and invariant rings," *IEEE Trans. Inf. Theory,* 1999, 45(4): 1194-1205.

[5] J. H. Conway, N. J. A. Sloane, Sphere Packings, Lattices and Groups, 3rd edn. Springer, New York (1998).

[6] M, Shi, Y. J. Choie, A. Sharma, and P. Sol´e. Codes and Modular Forms: A Dictionary. World Scientific, 2020.

[7] A. R. Calderbank, E. M. Rains, P. M. Shor, and N. J. A. Sloane, Quantum error correction via codes over GF(4), IEEE Trans. Inf. Theory, 1998, 44(4): 1369-1387.

[8] J.-L. Kim, "New extremal self-dual codes of lengths 36, 38, and 58," *IEEE Trans. Inform. Theory*, vol. 47, no. 1, pp. 386–393, 2001.

[9] J.-L. Kim and Y. Lee, "Euclidean and Hermitian self-dual MDS codes over large finite fields." J. Combin. Theory Ser. A 105(1):79-95, 2004.

[10] J.-L. Kim and Y. Lee, "Self-dual codes using the building-up construction," *IEEE Int. Symp. Inform. Theory*, Seoul, Korea (South), 2009, pp. 2400–2402.

[11] H. Montani, "Lagrangian and orthogonal splittings, quasitriangular Lie bialgebras and almost complex product structures," *J. Math. Phys.,* 64, 053505 (2023), arXiv:2208.09996v3.

[12] T. Chinburg and Y. Zhang, "Every binary self-dual code arises from Hilbert symbols," *Homology, Homotopy Appl.*, vol. 14, no. 2, pp. 189–196, 2012.

[13] M. Kreck and V. Puppe, "Involutions on 3-manifolds and self-dual, binary codes," *Homology, Homotopy Appl.*, vol. 10, no. 2, pp. 139–148, 2008.

[14] J. S. Leon, V. Pless, and N. J. A. Sloane, "Self-dual codes over GF(5)," *J. Combin. Theory Ser. A*, vol. 32, no. 2, pp. 178–194, 1982.

[15] M. Harada and P. R. J. Östergård, "On the classification of self-dual codes over $\mathbb{F}_5$," *Graphs Combin.*, vol. 19, no. 2, pp. 203–214, 2003.

[16] J.-L. Kim and W.-H. Choi, "Self-dual codes, symmetric matrices, and eigenvectors," *IEEE Access*, vol. 9, pp. 104294–104303, 2021.

[17] P. Abramenko and K. S. Brown, *Buildings: Theory and Applications*, Graduate Texts in Mathematics, vol. 248, Springer, 2008.

[18] V. Pless, "The number of isotropic subspaces in a finite geometry," *Atti Accad. Naz. Lincei Rend.*, vol. 39, pp. 418–421, 1965.

[19] L. Solomon, "The Steinberg character of a finite group with BN-pair," *Theory of Finite Groups*, Benjamin, 1969, pp. 213–221.

[20] J. Essert, "Buildings, group homology and lattices," *Groups Geom. Dyn.*, vol. 6, no. 2, pp. 247–278, 2012.

[21] W. C. Huffman and V. Pless, *Fundamentals of Error-Correcting Codes*, Cambridge University Press, 2003.